

アプリ総合演習ガイド

社員情報管理システム (EIMS)

目次

- 1. 演習概要
  - 1.1 社員一覧・検索・表示機能の仕様定義
  - 1.2 社員情報の登録機能
  - 1.3 社員情報の変更機能
  - 1.4 社員情報の削除機能
  - 1.5 開発・実行環境
- 2. 演習の進め方
  - 2.1 開発手順
  - 2.2 提供物について
  - 2.3 ドキュメントについて
- 3. ユースケース
  - 3.1 ユースケース図
  - 3.2 ユースケース仕様書
- 4. 画面一覧およびUIフロー図
  - 4.1 画面(HTML)一覧表
  - 4.2 全体UIフロー図
  - 4.3 機能別UIフロー図
- 5. 画面レイアウト図
  - 5.1 共通・検索機能
  - 5.2 登録機能
  - 5.3 変更機能
  - 5.4 削除機能
  - 5.5 入力チェック仕様
- 6. データベース仕様
  - 6.1 データベース概要
  - 6.2 テーブルの関係性 (実体関連図)
  - 6.3 テーブル定義
  - 6.4 初期データ仕様
- 7. クラス図 (分析レベル)
- 8. シーケンス図 (分析レベル)
  - 8.1 UC001:社員一覧を表示する
  - 8.2 UC002:社員情報を検索する
  - 8.3 UC003:社員情報を登録する
  - 8.4 UC004:社員情報を変更する

    - 8.5 UC005:社員情報を削除する
- 9. クラス図(設計レベル)
    - 9.1 UC001:社員一覧を表示する
    - 9.2 UC002:社員情報を検索する
    - 9.3 UC003:社員情報を登録する
    - 9.4 UC004:社員情報を変更する
    - 9.5 UC005:社員情報を削除する
    - 9.6 全体クラス図
- 10. シーケンス図 (設計レベル)
    - 10.1 URLマッピング (インターフェース仕様) 一覧
    - 10.2 UC001:社員一覧を表示する
    - 10.3 UC002:社員情報を検索する
    - 10.4 UC003:社員情報を登録する
    - 10.5 UC004:社員情報を変更する
    - 10.6 UC005:社員情報を削除する
- 11. コンポーネントテスト仕様(任意)
    - 11.1 Repository テストケース (任意)
    - 11.2 Controller テストケース (任意)
- 12. 共通メッセージ一覧
- 13. 発展課題（任意）
    - 13.1 二重ポスト（ダブルサブミット）対策
    - 13.2 社員一覧の並び替え（ソート）

アプリ総合演習ガイド

社員情報管理システム (EIMS)

1. 演習概要

この演習では、社員情報を管理する Web ベースのアプリケーションを構築する。 システム名は **EIMS (Employee Information Management System)** である。

1.1 社員一覧・検索・表示機能の仕様定義

人事部の管理者が社員の情報を俯瞰し、特定の社員を特定して詳細情報を確認するための機能群。

1.1.1 社員一覧表示機能

EIMSデータベースに登録されているすべての社員を一覧で表示する。

処理内容
社員テーブルの全レコードを取得し、一覧画面に表示する。

1.1.2 検索機能

各検索フォームからのリクエストに基づき、以下のロジックでデータベースを検索する。

検索方式	処理内容
社員番号検索	社員番号（主キー）を条件に検索を行う。該当する社員が存在する場合、直接「社員詳細画面」を表示する。
社員名検索	氏名（氏または名）をキーワードとし、部分一致検索を行う。検索結果は 「検索結果一覧画面」 に表示する。
部署検索	データベースから取得した部署情報のプルダウンから選択し、検索を行う。検索結果は 「検索結果一覧画面」 に表示する。

【社員名検索の入力例】

以下の入力例に基づき検索ロジックを実装すること。

- **例 1**: 「田」が入力された場合、**氏（last_name）** または **名（first_name）** のいずれかに「田」を含む社員を検索対象とする。

> **【補足】検索対象と入力のしかた**：この検索は **「氏（last_name）」列 または 「名（first_name）」列** に対する部分一致である。次のような入力では、該当者がいても 0 件になる点に注意すること（プレースホルダの「部分一致」もこの意味である）。

> - **氏と名をつなげたフルネーム**（例：「田中 太郎」「田中太郎」）は、「氏」「名」どちらの列にもその文字列自体は含まれないため一致しない。氏か名の**一部**（例：「田中」「太郎」）で検索する。
> - **カナ（氏カナ・名カナ）** は検索対象の列ではないため、カナで入力しても一致しない。

【共通の挙動ルール】

- **入力値の検証（各検索フォーム共通）**: 社員番号が未入力、社員名キーワードが `null`・空文字・空白のみ、または部署が未選択の場合は、社員の検索を行わず検索画面を再表示する。検索に対応するメッセージを `Model` の `message` に格納し、プルダウン用の部署一覧を `departments` に再設定する。画面には「社員番号を入力してください。」「社員名を入力してください。」「部署を選択してください。」のいずれかを表示する。
- **社員名の文字数**: 検索キーワードの文字数上限は設けない。ただし、未入力・空文字・空白のみは検索しない。前後の空白は除去せず、入力された値をそのまま検索条件に使う。登録・変更時の氏・名の10文字制限とは別の規則である。具体的な入力条件は5.5.1節、文言は12章を参照する。
- **検索処理の分離**: 社員番号・社員名・部署の各フォームは専用URLへ送信し、Controllerも `searchByEmpNo`・`searchByEmpName`・`searchByDeptNo` に分ける。他の検索項目が送られても別の検索へ切り替えない。

1.1.3 一覧画面および社員詳細画面

一覧（社員一覧および検索結果一覧）と詳細画面の表示ルールを以下に定義する。

- **一覧画面（共通ルール）**:
  - 以下の 4 項目をテーブル形式で表示する。
    1. **社員番号**
    2. **氏名（カナ）**：「氏 名（氏カナ 名カナ）」の形式で連結して表示する。
    3. **性別**： DB値が 1 なら「男性」、2 なら「女性」と置換して表示する。
    4. **部署名**
  - **氏名（カナ）をクリック**することで、対象社員の「社員詳細画面」へ遷移できる導線を設けること。
  - 検索結果が 0 件の場合は、テーブルを表示せず「検索条件に一致する社員は見つかりませんでした。」というメッセージを提示すること。
  - 「**メニューに戻る**」 ボタンと、**検索画面への導線**を配置すること。検索画面への導線は、**社員一覧画面では「検索画面へ」**（前進）、**検索結果一覧画面では「検索画面に戻る」**（戻り）とする（UC001 alt3-3／UC002 の代替フローに対応）。
- **社員詳細画面**:
  - 社員番号、氏、名、氏（カナ）、名（カナ）、性別、部署名を表示する。
  - この画面を起点とし、後のステップで実装する「変更」「削除」の機能へ繋げる。
  - **「一覧に戻る」「検索画面に戻る」「メニューに戻る」ボタン** を配置すること。なお「一覧に戻る」は社員一覧画面（全件表示）へ戻る動線であり、直前の検索条件は保持しない。
- **共通の表示ルール**:
  - 氏名、カナを表示する際は、氏と名の間を**半角スペース**で連結すること。

1.1.4 検索・表示における実装方針と簡略化指針

チームの進捗状況に合わせ、以下のいずれかの方針を選択すること。

標準実装と講師用解答では、1.1.2節・5.5.1節の入力チェックとメッセージ表示を必須とする。簡略版でも、検索の未入力チェックとメッセージ表示は省略しない。

項目	標準的な実装（推奨）	簡略化された実装
バリデーション	未入力チェックとメッセージ表示を実装	標準実装と同じ（省略しない）
0件時の制御	メッセージ表示による分岐処理を実装	特になし（空の表を表示）
氏名検索範囲	**氏** または **名** の 2 項目 OR 検索	**氏（漢字）** のみの単一項目検索
部署検索方式	**部署名** をプルダウンで選択	**部署コード** をテキストで入力
一覧の部署表示	**部署名**（名称）をテーブルに表示	**部署コード**（数値）をそのまま表示

1.2 社員情報の登録機能

新しい社員の情報をデータベースへ登録するための機能群。

1.2.1 登録機能

登録画面から送信された社員情報を検証し、データベースへ保存する。

- **入力項目**:

項目	入力方式	バリデーション条件
氏	テキスト入力	必須、10文字以内
名	テキスト入力	必須、10文字以内
氏（カナ）	テキスト入力	必須、20文字以内
名（カナ）	テキスト入力	必須、20文字以内
パスワード	パスワード入力	必須、4文字以上16文字以内
性別	ラジオボタン（男性/女性）	必須、初期状態は未選択
部署	セレクトメニュー（部署テーブルより取得）	必須

- **社員番号**: データベースの自動採番機能（AUTO_INCREMENT）を使用する。入力項目には含めない。
- **バリデーションエラー時**: プルダウン用の部署一覧をデータベースから再取得した上で登録画面に戻り、エラーメッセージを表示する。パスワード以外の入力済みの値は保持すること。パスワード欄は空欄にし、再入力を求める。

1.2.2 登録画面・登録確認画面・登録完了画面

登録プロセスにおける各画面の仕様を以下に定義する。

- **登録画面**:
  - 上記 1.2.1 の入力項目を配置し、ユーザーからの入力を受け付ける。
  - **「確認画面へ」ボタン** および **「メニューに戻る」ボタン** を配置すること。
- **登録確認画面**:
  - 登録画面で入力された内容を、読み取り専用で一覧表示する。
  - 「氏」「名」「氏（カナ）」「名（カナ）」は、個別の項目として表示してよい。
  - パスワードは **「**** **（非表示）」** のように、実際の値が分からない形式でマスク表示すること。マスク文字の種類や文字数は問わない。
  - **「登録確定」ボタン** および **「修正する」ボタン** を配置すること。
  - 「修正する」ボタン押下時は、プルダウン用の部署一覧を再取得し、パスワード以外の入力データが保持された状態で登録画面へ戻ること。パスワード欄は空欄にし、再入力を求める。
- **登録完了画面**:

- 登録が正常に完了した旨のメッセージを表示する。
- **「続けて登録する」ボタン** および **「メニューに戻る」ボタン** を配置すること。
- 「続けて登録する」ボタン押下時は、プルダウン用の部署一覧を再取得し、全項目が空欄の登録画面を表示すること。

1.2.3 登録における実装方針と簡略化指針

チームの進捗状況に合わせ、以下のいずれかの方針を選択すること。

項目	標準的な実装（推奨）	簡略化された実装
登録フロー	入力 →**確認画面** → 完了	入力 → 完了（確認画面なし）
「修正する」ボタンの挙動	パスワード以外の入力値を保持して登録画面へ戻る。パスワードは再入力する	登録画面へ戻る（ただし入力値は空欄になる）
性別の入力方式	ラジオボタン（男性/女性）	テキスト入力（数値 1 または 2）
部署の入力方式	セレクトメニュー（部署名を選択）	テキスト入力（部署コードを入力）
バリデーション	全項目の入力チェックを実装	バリデーションなし

1.3 社員情報の変更機能

既存の社員情報を変更し、データベースへ上書き保存するための機能群。 変更対象の社員は、社員詳細画面から選択する。

1.3.1 変更機能

変更画面から送信された社員情報を検証し、データベースの既存レコードを上書き保存する。

- **変更対象の特定**: 社員詳細画面に表示されている社員番号（主キー）を基に、変更対象を特定する。
- **変更可能な項目**:

項目	入力方式	バリデーション条件
氏	テキスト入力	必須、10文字以内
名	テキスト入力	必須、10文字以内
氏（カナ）	テキスト入力	必須、20文字以内
名（カナ）	テキスト入力	必須、20文字以内
パスワード	パスワード入力	必須、4文字以上16文字以内
性別	ラジオボタン（男性/女性）	必須
部署	セレクトメニュー（部署テーブルより取得）	必須

- **社員番号**: 変更不可。画面上に表示するが、入力フィールドとしては提供しない。
- **初期表示**: 変更画面を開いた時点で、パスワード以外の現在の登録内容が各入力欄にセットされた状態で表示すること。パスワード欄は空欄にし、入力を求める。
- **バリデーションエラー時**: プルダウン用の部署一覧をデータベースから再取得した上で変更画面に戻り、エラーメッセージを表示する。パスワード以外の入力済みの値は保持すること。パスワード欄は空欄にし、再入力を求める。

1.3.2 変更画面・変更確認画面・変更完了画面

変更プロセスにおける各画面の仕様を以下に定義する。

- **変更画面**:
  - 社員番号を読み取り専用で表示し、その他の項目を編集可能な状態で表示する。
  - パスワード以外の入力欄には、現在の登録データを初期値として表示すること。パスワード欄は空欄とし、変更時にはパスワードを入力すること。
  - **「確認画面へ」ボタン** および **「詳細に戻る」ボタン** を配置すること。
- **変更確認画面**:
  - 変更後の内容を、読み取り専用で一覧表示する。
  - 「氏」「名」「氏（カナ）」「名（カナ）」は、個別の項目として表示してよい。

    - パスワードは 「**** (**非表示**)」 のように、実際の値が分からない形式でマスク表示すること。マスク文字の種類や文字数は問わない。
    - 「**変更確定**」**ボタン** および 「**修正する**」**ボタン** を配置すること。
    - 「修正する」ボタン押下時は、プルダウン用の部署一覧を再取得し、パスワード以外の入力データが保持された状態で変更画面へ戻ること。パスワード欄は空欄にし、再入力を求める。

- **変更完了画面**:
    - 変更が正常に完了した旨のメッセージを表示する。
    - 「**メニューに戻る**」**ボタン** および 「**検索画面に戻る**」**ボタン** を配置すること。

1.3.3 変更における実装方針と簡略化指針

チームの進捗状況に合わせ、以下のいずれかの方針を選択すること。

項目	標準的な実装（推奨）	簡略化された実装
変更フロー	変更入力 →**確認画面** → 完了	変更入力 → 完了（確認画面なし）
「修正する」ボタンの挙動	パスワード以外の入力値を保持して変更画面へ戻る。パスワードは再入力する	変更画面へ戻る（ただし入力値は空欄になる）
性別の入力方式	ラジオボタン（男性/女性）	テキスト入力（数値 1 または 2）
部署の入力方式	セレクトメニュー（部署名を選択）	テキスト入力（部署コードを入力）
バリデーション	全項目の入力チェックを実装	バリデーションなし

1.4 社員情報の削除機能

指定された社員情報をデータベースから削除（物理削除）するための機能群。 削除対象の社員は、社員詳細画面から選択する。

1.4.1 削除機能

社員詳細画面に表示されている社員番号（主キー）を基に、対象レコードをデータベースから削除する。

- **削除の起点**: 社員詳細画面の「削除」ボタンより削除プロセスを開始する。
- **削除方式**: データベースからレコードを完全に削除する（物理削除）。
- **削除対象が存在しない場合**: 削除できない旨を表示すること。

1.4.2 削除確認画面・削除完了画面

削除プロセスにおける各画面の仕様を以下に定義する。

- **削除確認画面**:
  - 削除対象の社員情報（社員番号、氏、名、氏（カナ）、名（カナ）、性別、部署名）を読み取り専用で表示する。
  - 「この社員を削除しますか？」のような確認メッセージを表示すること。
  - **「削除確定」ボタン** および **「詳細に戻る」ボタン** を配置すること。
- **削除完了画面**:
  - 削除が正常に完了した旨のメッセージを表示する。
  - **「メニューに戻る」ボタン** および **「検索画面に戻る」ボタン** を配置すること。

1.4.3 削除における実装方針と簡略化指針

チームの進捗状況に合わせ、以下のいずれかの方針を選択すること。

項目	標準的な実装（推奨）	簡略化された実装
削除対象の確認表示	削除対象の社員情報を表示	表示なし
削除対象が存在しない場合の制御	削除できない旨を表示	特になし

1.5 開発・実行環境

本演習で構築するシステムの開発および実行環境は以下の通りとする。

カテゴリ	技術・バージョン
Java	17
Framework	Spring Boot 4.0.2
Build Tool	Maven
Database	MySQL 8
Template Engine	Thymeleaf
CSS Framework	Bootstrap 5

> **データベースとエンティティの整合について（重要）** 本演習のデータベース構造（テーブル定義）は、提供される「データベース構築ファイル」であらかじめ作成されます。スターターキットの `application.properties` には `spring.jpa.hibernate.ddl-auto=validate` が設定されており、アプリ起動時に **エンティティ（`@Entity` クラス）の定義が DB のテーブル定義（「6.3 テーブル定義」参照）と一致するか** を検証します。
>
> ⚠ **エンティティと DB の表定義が一致しないと、アプリケーションは起動せず、起動時にエラーで停止します。** これは列名・型の誤りや `@Column(name = ...)` の付け忘れといったマッピングのミスを早期に発見するための設定です。起動に失敗したときは、エラーメッセージを読み、該当するエンティティと「6.3 テーブル定義」を突き合わせて修正してください。

2. 演習の進め方

2.1 開発手順

以下の手順に従い、システムを構築すること。機能ごとに以下の作業を行う。

順序	作業	説明	成果物
1	要件定義の内容確認	以下の各種ドキュメントを元に、要件を理解する。 ・ユースケース図 ・ユースケース仕様書 ・UI フロー図 ・画面レイアウト図	※成果物の作成は不要。提供された資料を用いる。
2	設計の内容確認	設計レベルのクラス図の記述内容を理解する。	※成果物の作成は不要。提供された資料を用いる。
3	プログラミング	設計モデルどおりにコーディングする。	ソースコード
4	テスト	要件定義資料を基にシステムテストケースを自ら作成し、システムテストを実施する。 余裕がある場合は、11章で提供される単体テストケースを参考に、コンポーネントテスト（JUnit）を実装してもよい。	システムテストケース仕様書 テスト不具合報告書 動作するプログラム

テストの範囲・観点

本演習で作成・実施するテストは、本ガイドで定義した仕様を対象とする：各ユースケースの基本／代替フロー、5.5 入力チェック仕様（必須・文字数）、12章 共通メッセージ、画面遷移。

一方、以下は本演習のテスト対象に含めなくてよい（実装によっては異常終了することがあるが、本演習では許容する）：

- 入力方式（ラジオボタン／プルダウン）で値が制限される項目への**値域外・改ざん送信**（例：性別＝3、存在しない部署番号の送信）
- **URL を直接操作した不正アクセス**（例：存在しない社員番号への直接遷移）
- 完了画面の更新による**二重送信**（※二重ポストは 13 章の発展課題で扱う）

2.2 提供物について

本演習では、以下のものが事前に提供される。

提供物	説明
スターターキットプロジェクト	必要なライブラリ（依存関係）およびリソースファイル（イメージファイル等）が設定済みの Spring Boot プロジェクト。このプロジェクトをベースに開発を開始すること。なお Bootstrap は CDN から読み込む構成で、社内ネットワーク等で CDN にアクセスできない場合に備え、ローカル版（`static/css/bootstrap.min.css`）も同梱している。
データベース構築ファイル	テーブル作成スクリプトおよび初期データの投入スクリプト。本番用データベース `eimsdb` は `EIMS_DBSET`、テスト用データベース `eimsdb_test` は `EIMS_TEST_DBSET` を使用し、2つのデータベースを構築すること。講師の指示に従い、データベースのセットアップを行うこと。
各種設計ドキュメント	本ドキュメント内に、ユースケース図、ユースケース仕様書、UIフロー図、画面レイアウト図、クラス図（分析・設計レベル）、シーケンス図を収録している。
テストケース仕様書フォーマット	テスト工程で使用する書式ファイル（.xls）。

2.3 ドキュメントについて

各工程で作成・利用する成果物は以下の表の通り。

工程	成果物	社員情報検索	社員情報登録	社員情報削除	社員情報変更
要件定義	ユースケース図	※			
要件定義	ユースケース仕様書	※	※	※	※
要件定義	UI フロー図	※	※	※	※
要件定義	画面レイアウト図	※	※	※	※
設計	画面（HTML）一覧表	※			
設計	URLマッピング一覧表	※			
設計	データベース仕様	※			
分析	クラス図（分析レベル）	※	※	※	※
分析	シーケンス図（分析レベル）	※	※	※	※
設計	クラス図（設計レベル）	※	※	※	※
設計	シーケンス図（設計レベル）	※	※	※	※
設計	コンポーネントテスト仕様（任意）	※			
実装	ソースコード	○	○	○	○
テスト	システムテストケース仕様書.xls	○	○	○	○
テスト	テスト不具合報告書.xls	○	○	○	○

表の見方は次の通り。

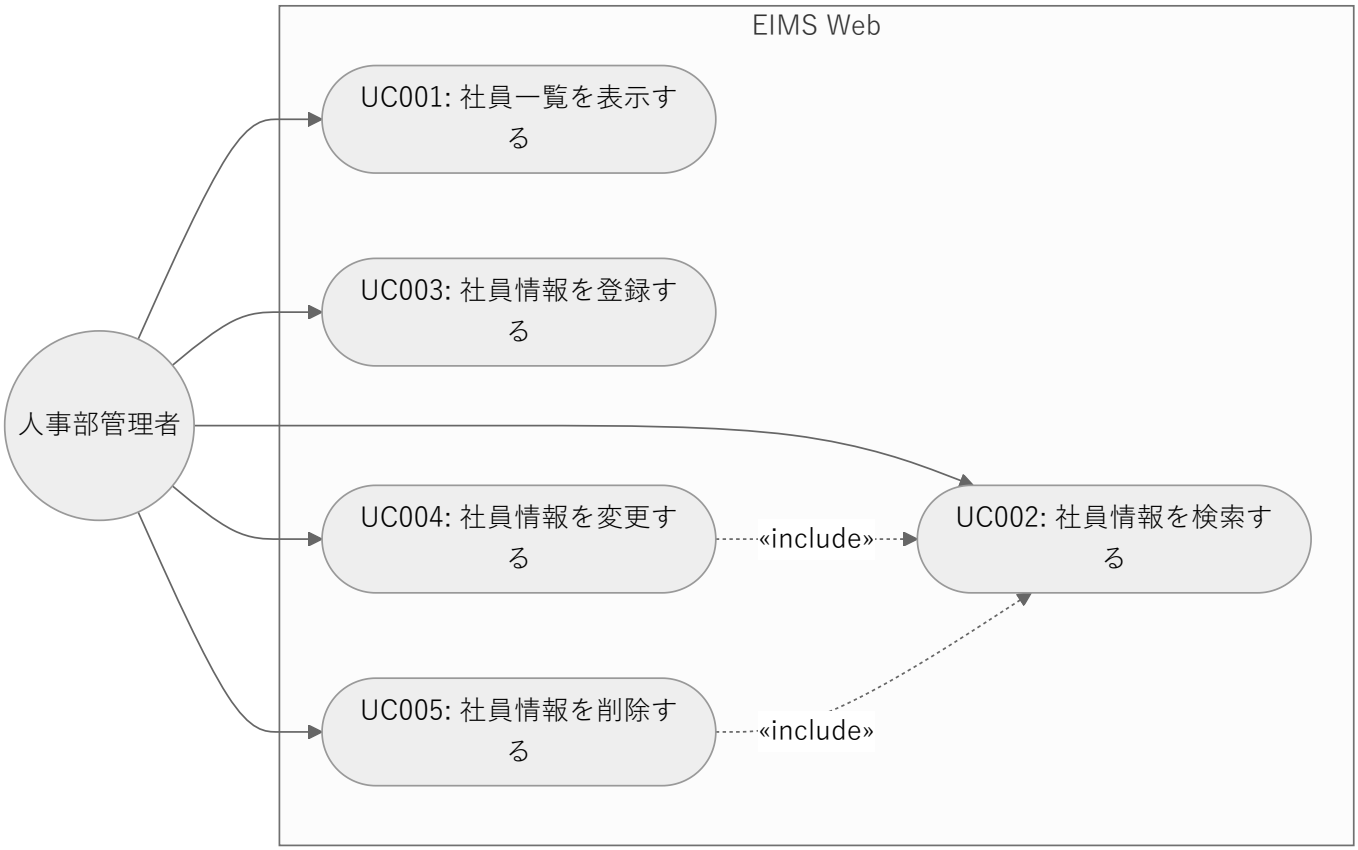
- 「※」 ・・・ 本ドキュメントに収録、または提供される作成済みの資料を利用する。
- 「〇」 ・・・ 自身で作成する。作成にあたっては、提供されるフォーマット（書式）を利用する。

また、データベースのテーブルレイアウトは確定済みである。「6. データベース仕様」を参照すること。

3. ユースケース

3.1 ユースケース図

社員情報管理



- **UC004（変更）** および **UC005（削除）** は、対象社員を特定するために **UC002（検索）** を包含（include）する。

3.2 ユースケース仕様書

UC001: 社員一覧を表示する

項目	内容
ユースケースID	UC001
ユースケース名	社員一覧を表示する
目的	EIMSデータベースに登録されている全社員の一覧を表示する。
アクター	人事部管理者

事前条件:

- EIMS データベースに社員テーブルと部署テーブルが作成されていること。
- EIMS データベースが正常に起動していること。

基本フロー:

1. [人事部管理者]は、システムに「社員一覧」を指示する。
2. システムは、EIMSデータベースから全社員情報を取得し、社員一覧画面を表示する。
3. [人事部管理者]は、表示された一覧を確認し、ユースケースを終了する。(代替フロー alt3-1, alt3-2, alt3-3 参照)

代替フロー:

- **alt3-1.** 基本フロー3で[人事部管理者]が一覧の氏名+カナをクリックした場合、システムは対象社員の「社員詳細画面」を表示する。
- **alt3-2.** 基本フロー3で[人事部管理者]はトップページに遷移することができる。
- **alt3-3.** 基本フロー3で[人事部管理者]は検索画面に遷移することができる。

事後条件:

- 社員一覧画面が表示されていること。
- 一覧の表は「社員番号」「氏名+カナ」「性別」「部署名」の4列で構成され、「氏」・「名」および「氏(カナ)」・「名(カナ)」は半角スペース区切りで連結されていること。

特記事項: 特になし。

UC002: 社員情報を検索する

項目	内容
ユースケースID	UC002
ユースケース名	社員情報を検索する
目的	社員番号による検索と、氏・名の部分一致検索と、部署名検索を行い、該当する社員情報の一覧を表示する。
アクター	人事部管理者

事前条件:

- EIMS データベースに社員テーブルと部署テーブルが作成されていること。
- EIMS データベースが正常に起動していること。

基本フロー:

1. [人事部管理者]は、システムに「社員の検索」を指示する。
2. システムは、EIMSデータベースからプルダウン用の部署一覧を取得し、検索画面を表示する。
3. [人事部管理者]は、社員番号・社員名・部署名のいずれかの入力条件で検索を指示する。
4. システムは、送信先URLに対応する検索条件を検証し、有効な場合に[EIMSデータベース]の検索を行い、検索結果一覧を表示する。（代替フロー alt4-1, alt4-2, alt4-3 参照）
5. [人事部管理者]は、表示された検索結果を確認し、ユースケースを終了する。（代替フロー alt5-1, alt5-2, alt5-3 参照）

代替フロー:

- **alt4-1.** 基本フロー4で該当する社員がいない（検索結果0件）場合、システムは、「検索条件に一致する社員は見つかりませんでした。」を検索結果画面に表示する。
- **alt4-2.** 基本フロー3で社員番号による検索の場合、該当する社員がいれば、検索結果一覧を経由せず直接「社員詳細画面」を表示する。該当しない場合はalt4-1に従う。
- **alt4-3.** 基本フロー4で未入力・未選択を検出した場合、システムは社員の検索を行わず、部署一覧を再取得して検索画面に戻る。5.5.1節の対応メッセージを表示し、再入力を受け付ける。
- **alt5-1.** 基本フロー5で[人事部管理者]が再検索を指示した場合は、基本フロー2に戻り、システムは部署一覧を取得し、検索画面を表示する。
- **alt5-2.** 基本フロー5で[人事部管理者]はトップページに遷移することができる。
- **alt5-3.** 基本フロー5で[人事部管理者]が検索結果一覧の氏名+カナをクリックした場合、システムは対象社員の「社員詳細画面」を表示する。

事後条件:

- 有効な条件で検索を行った場合は、検索結果画面が表示されること。ただし、社員番号検索で対象が見つかった場合は、社員詳細画面が表示されること。
- 検索結果が0件の場合は、その旨メッセージが画面に表示されること。
- 社員名検索の場合は、社員テーブルの「氏」または「名」のどちらかに検索条件を含む社員の情報が検索結果として表示されていること。

- 検索結果の表は「社員番号」「氏名+カナ」「性別」「部署名」の4列で構成され、「氏」・「名」および「氏(カナ)」・「名(カナ)」は半角スペース区切りで連結されていること。

特記事項: 特になし。

UC003: 社員情報を登録する

項目	内容
ユースケースID	UC003
ユースケース名	社員情報を登録する
目的	氏、名、氏（カナ）、名（カナ）、性別、部署、パスワードを入力して、一人分の社員情報を登録する。社員番号はシステムが自動採番する。
アクター	人事部管理者

事前条件:

- EIMS データベースに社員テーブルと部署テーブルが作成されていること。
- EIMS データベースが正常に起動していること。

基本フロー:

1. [人事部管理者]は、システムに「社員の登録」を指示する。
2. システムは、EIMSデータベースからプルダウン用の部署一覧を取得し、社員情報登録画面を表示する。
3. [人事部管理者]は、社員情報登録画面に登録する社員の情報を入力し、登録を指示する。（代替フロー alt3-1 参照）
4. システムは、入力された社員情報の妥当性を検証し、問題がない場合は登録確認画面を表示する。（代替フロー alt4-1 参照）
5. [人事部管理者]は、登録する社員の情報を確認し、問題がなければ登録を確定する。（代替フロー alt5-1 参照）
6. システムは、登録する社員の社員番号を生成し、社員情報を[EIMSデータベース]へ登録する。正常に登録できた場合は社員情報登録完了画面を表示する。
7. [人事部管理者]は、登録完了画面を確認し、ユースケースを終了する。（代替フロー alt7-1 参照）

代替フロー:

- **alt3-1.** 基本フロー3で、[人事部管理者]は、社員登録を中止することができる。
    1. [人事部管理者]は、登録中止を指示する。
    2. システムは、トップページを表示し、ユースケースを終了する。

- **alt4-1.** 基本フロー4で、社員情報に不正がある場合、システムは、基本フロー2に戻り、部署一覧を再取得した上で、パスワード以外の入力値と不正箇所を指摘するメッセージを表示して修正を促す。パスワード欄は空欄にし、再入力を求める。
- **alt5-1.** 基本フロー5で、[人事部管理者]は登録を取り消すことができる。
    1. [人事部管理者]は修正を指示する。
    2. システムは、部署一覧を再取得し、パスワード以外の入力値を保持した状態で社員情報登録画面を表示する。パスワード欄は空欄にし、再入力を求める。

- **alt7-1.** 基本フロー7で、[人事部管理者]は、追加登録を指示することができる。
    1. [人事部管理者]が追加登録を指示する。

2. システムは、基本フロー2に戻り、部署一覧を取得し、全ての項目が空欄の状態で社員情報登録画面を表示する。

事後条件:

- 登録が完了した場合は、登録完了画面が表示されていること。
- 登録が完了した場合は、EIMSデータベースに入力した社員のデータが保存されていること。

特記事項: 特になし。

UC004: 社員情報を変更する

項目	内容
ユースケースID	UC004
ユースケース名	社員情報を変更する
目的	既存の社員情報を変更し、EIMSデータベースを更新する。社員番号は変更不可とする。
アクター	人事部管理者

事前条件:

- EIMS データベースに社員テーブルと部署テーブルが作成されていること。
- EIMS データベースが正常に起動していること。
- UC002（検索）により対象社員が特定され、社員詳細画面が表示されていること。

基本フロー:

1. [人事部管理者]は、社員詳細画面から「社員の変更」を指示する。
2. システムは、EIMSデータベースからプルダウン用の部署一覧を取得し、現在の登録データをセットした社員情報変更画面を表示する。ただし、パスワード欄には値を表示せず、入力を求める。
3. [人事部管理者]は、変更内容を入力し、変更を指示する。（代替フロー alt3-1 参照）
4. システムは、入力された社員情報の妥当性を検証し、問題がない場合は変更確認画面を表示する。（代替フロー alt4-1 参照）
5. [人事部管理者]は、変更内容を確認し、問題がなければ変更を確定する。（代替フロー alt5-1 参照）
6. システムは、[EIMSデータベース]の対象レコードを更新し、変更完了画面を表示する。
7. [人事部管理者]は、変更完了画面を確認し、ユースケースを終了する。

代替フロー:

- **alt3-1.** 基本フロー3で、[人事部管理者]は、変更を中止することができる。
    1. [人事部管理者]は、変更中止を指示する。
    2. システムは、社員詳細画面に戻り、ユースケースを終了する。
- **alt4-1.** 基本フロー4で、社員情報に不正がある場合、システムは、基本フロー2に戻り、部署一覧を再取得した上で、パスワード以外の入力値と不正箇所を指摘するメッセージを表示して修正を促す。パスワード欄は空欄にし、再入力を求める。
- **alt5-1.** 基本フロー5で、[人事部管理者]は変更を取り消すことができる。
    1. [人事部管理者]は修正を指示する。
    2. システムは、部署一覧を再取得し、パスワード以外の入力値を保持した状態で社員情報変更画面を表示する。パスワード欄は空欄にし、再入力を求める。
- **alt7-1.** 基本フロー7で[人事部管理者]はトップページに遷移することができる。
- **alt7-2.** 基本フロー7で[人事部管理者]は検索画面に遷移することができる。

事後条件:

- 変更が完了した場合は、変更完了画面が表示されていること。
- 変更が完了した場合は、EIMSデータベースの対象社員のデータが更新されていること。

特記事項: 社員番号は変更不可。画面上に表示するが、入力フィールドとしては提供しない。

UC005: 社員情報を削除する

項目	内容
ユースケースID	UC005
ユースケース名	社員情報を削除する
目的	指定された社員の情報をEIMSデータベースから物理削除する。
アクター	人事部管理者

事前条件:

- EIMS データベースに社員テーブルと部署テーブルが作成されていること。
- EIMS データベースが正常に起動していること。
- UC002（検索）により対象社員が特定され、社員詳細画面が表示されていること。

基本フロー:

1. [人事部管理者]は、社員詳細画面から「社員の削除」を指示する。
2. システムは、削除対象の社員情報を表示した削除確認画面を表示する。
3. [人事部管理者]は、削除対象を確認し、削除を確定する。(代替フロー alt3-1 参照)
4. システムは、[EIMSデータベース]から対象レコードを削除し、削除完了画面を表示する。
5. [人事部管理者]は、削除完了画面を確認し、ユースケースを終了する。

代替フロー:

- **alt3-1.** 基本フロー3で、[人事部管理者]は削除を中止することができる。
    1. [人事部管理者]は、削除中止を指示する。
    2. システムは、社員詳細画面に戻り、ユースケースを終了する。
- **alt5-1.** 基本フロー5で[人事部管理者]はトップページに遷移することができる。
- **alt5-2.** 基本フロー5で[人事部管理者]は検索画面に遷移することができる。

事後条件:

- 削除が完了した場合は、削除完了画面が表示されていること。
- 削除が完了した場合は、EIMSデータベースから対象社員のデータが削除されていること。

特記事項: 特になし。

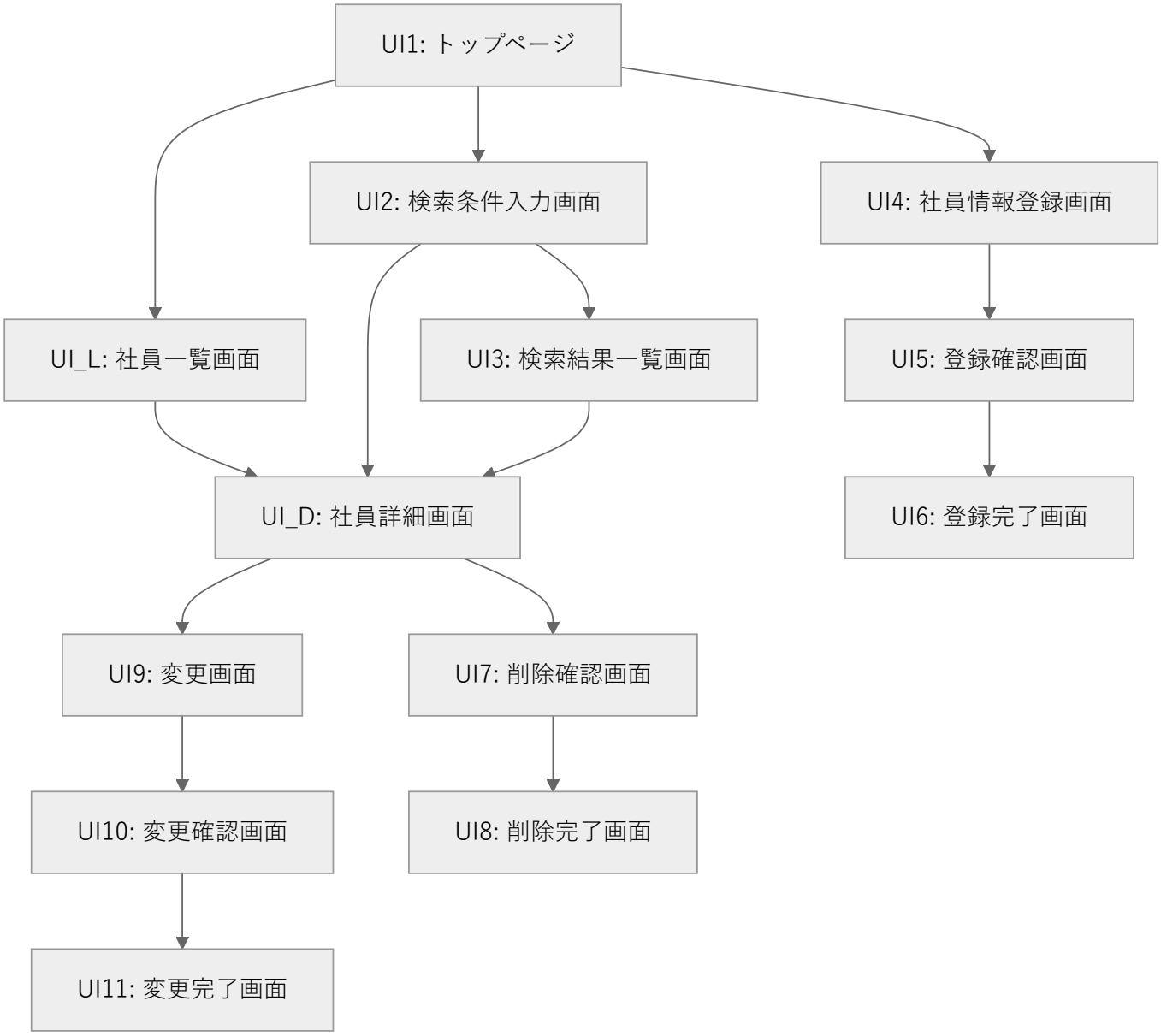
4. 画面一覧およびUIフロー図

4.1 画面（HTML）一覧表

本システムで作成・使用する画面と、対応するHTMLファイル（Thymeleafテンプレート）の一覧を以下に定義する。

画面ID	画面名	HTMLファイル名	概要
UI1	トップページ	index.html	各機能への導線となるメニュー画面。
UI2	検索条件入力画面	search.html	社員情報を検索するための条件（社員番号、氏名、部署）を入力する画面。
UI3	検索結果一覧画面	search_result.html	氏名・部署による検索にヒットした社員の一覧を表示する画面。
UI4	社員情報登録画面	input.html	新規に社員情報を登録するための入力画面。
UI5	登録確認画面	input_confirm.html	登録前に入力内容を確認する画面。
UI6	登録完了画面	input_complete.html	登録処理が正常に完了したことを通知する画面。
UI7	削除確認画面	delete_confirm.html	削除前に本当に対象社員を削除してよいか確認する画面。
UI8	削除完了画面	delete_complete.html	削除処理が正常に完了したことを通知する画面。
UI9	変更画面	change.html	既存の社員情報を変更するための入力画面。
UI10	変更確認画面	change_confirm.html	変更前に入力内容を確認する画面。
UI11	変更完了画面	change_complete.html	変更処理が正常に完了したことを通知する画面。
UI_L	社員一覧画面	employee_list.html	全社員のリストを表示する画面。
UI_D	社員詳細画面	employee_detail.html	特定の社員の詳細情報を表示する画面。

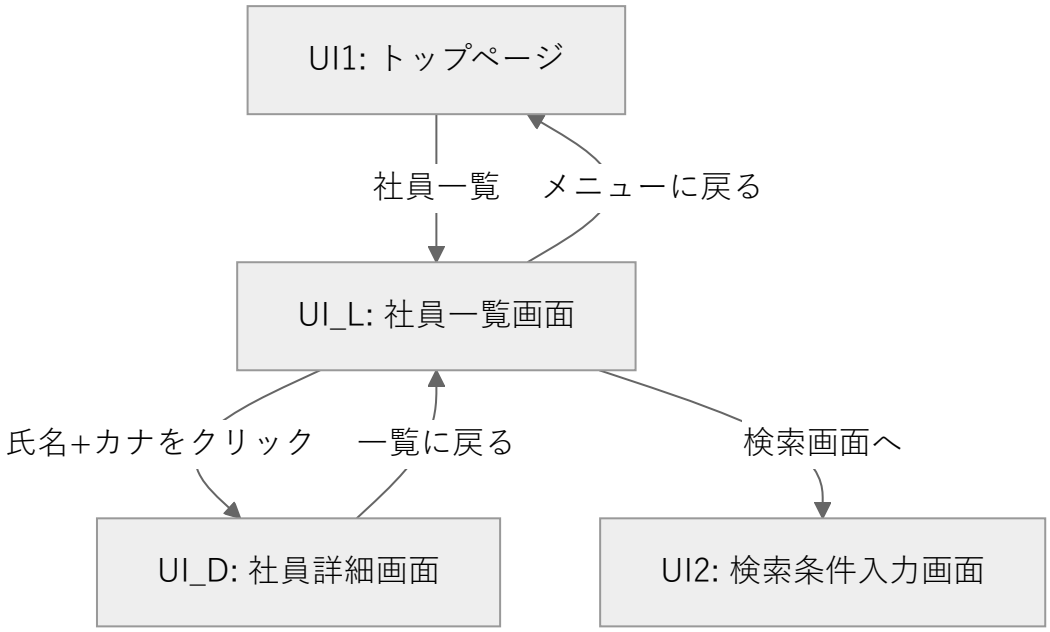
4.2 全体UIフロー図



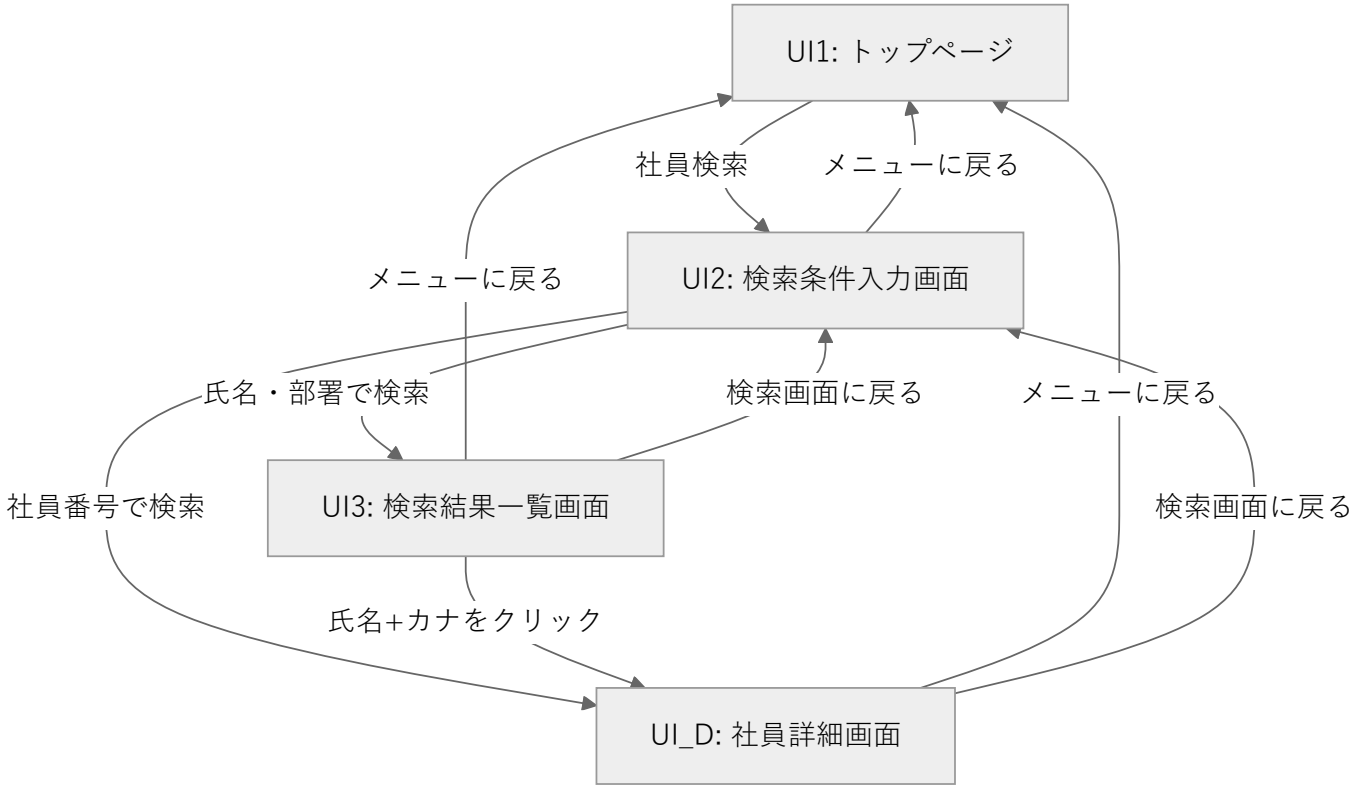
※図の煩雑化を防ぐため、各画面から「トップページ」や「検索画面」へ戻るルートは全体図では省略しています。各機能別のフロー図を参照してください。 ※「検索画面に戻る」「修正する」等による検索画面や入力画面への遷移は、ブラウザバック等ではなく必ずControllerへリクエストを送信し、毎回プルダウン用の部署一覧を再取得する設計とします。

4.3 機能別UIフロー図

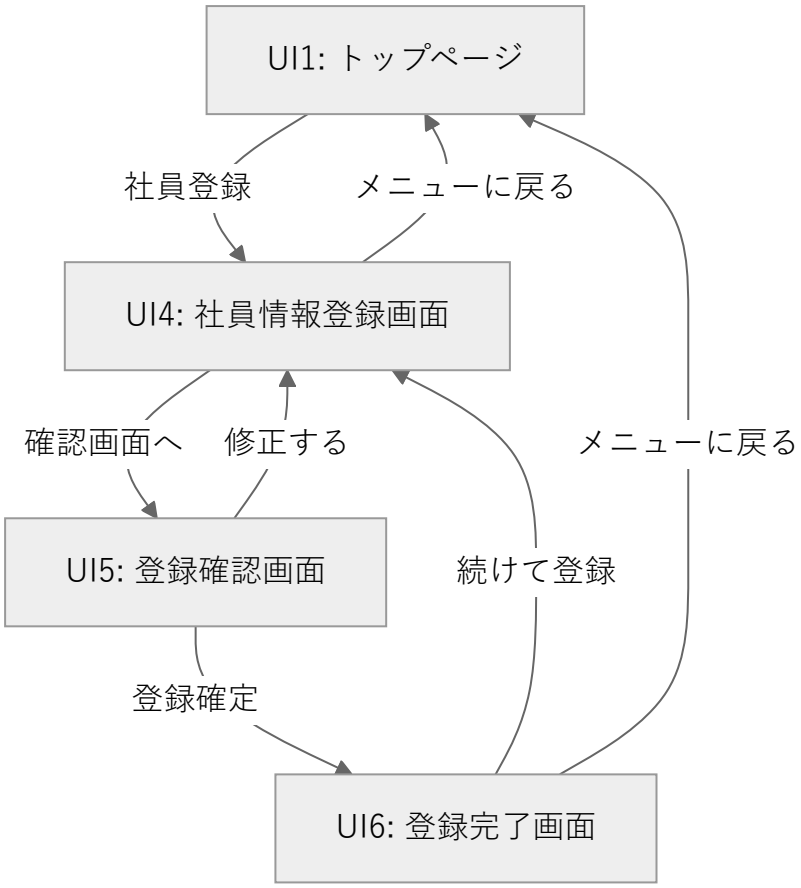
UC001: 社員一覧を表示する



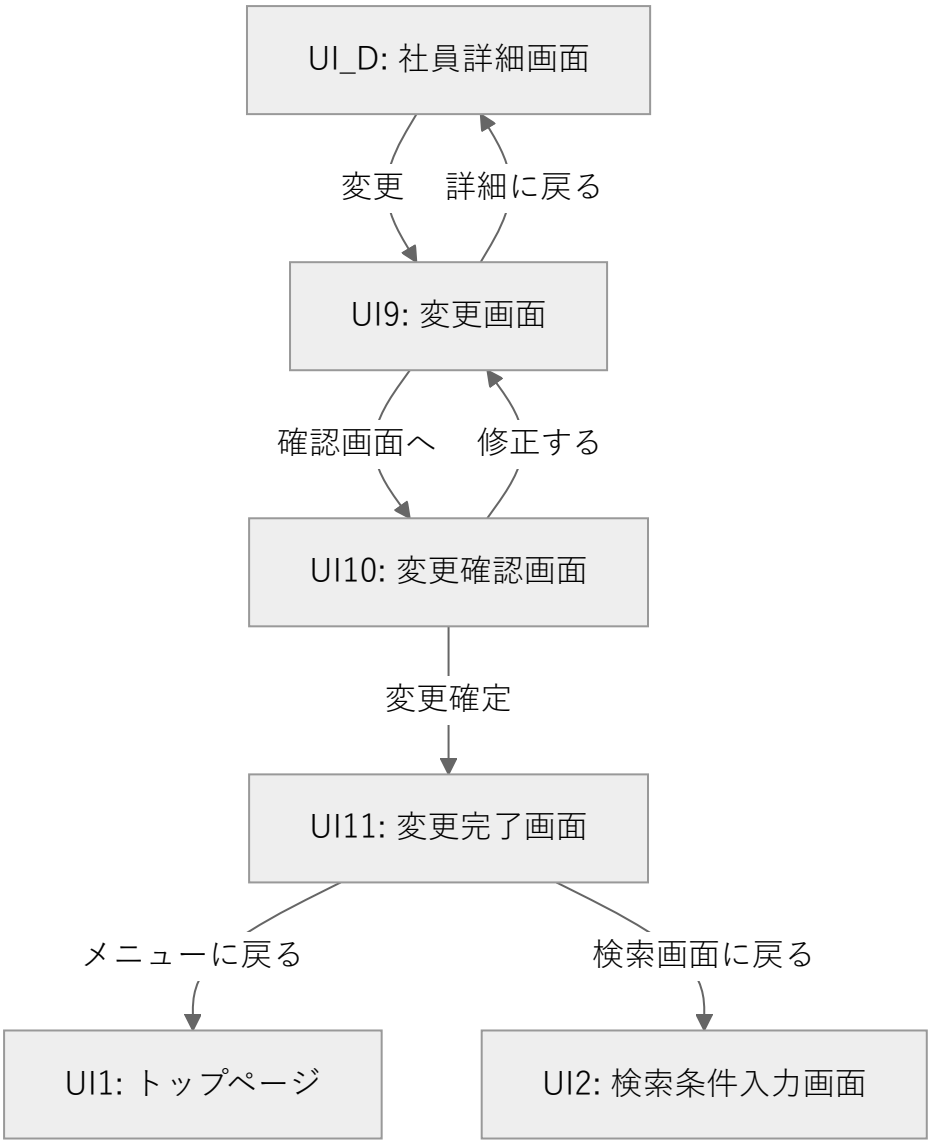
UC002: 社員情報を検索する



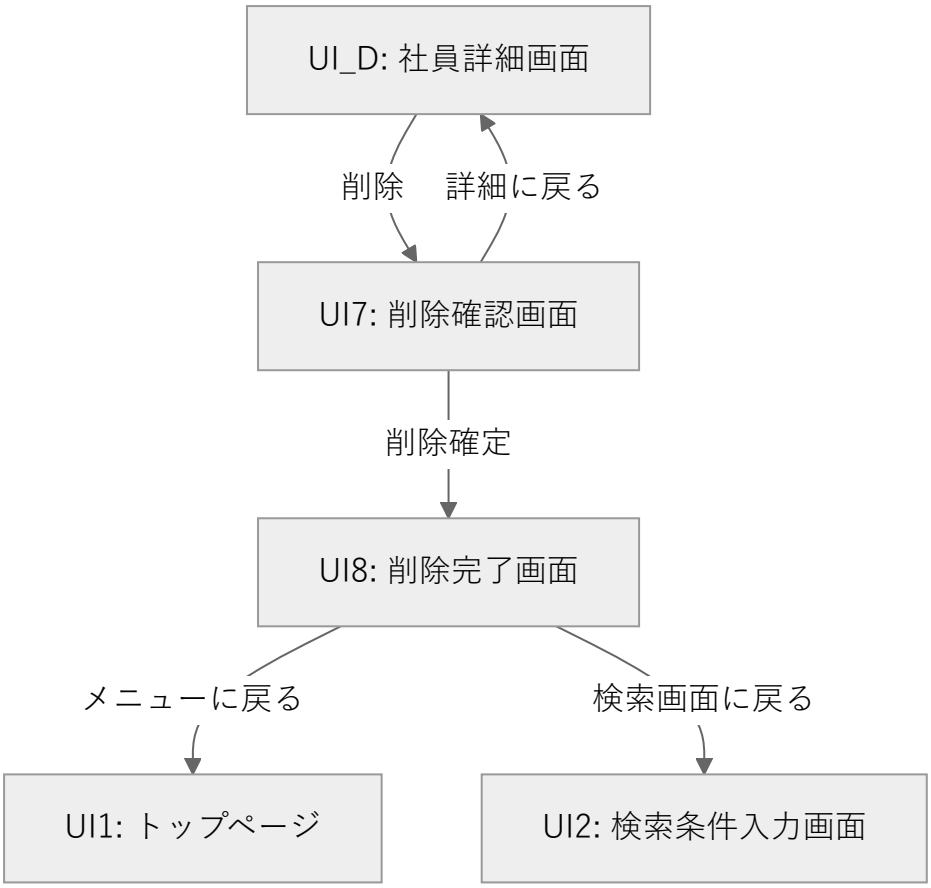
UC003: 社員情報を登録する



UC004: 社員情報を変更する



UC005: 社員情報を削除する



5. 画面レイアウト図

各画面のレイアウト仕様を以下に定義する。

Note

本設計書に記載されている画面レイアウト図は、視認性向上のために Bootstrap5 を適用した状態を参考として掲載しています。 共通機能の実装課題においては、**デザイン（CSS/レイアウト）の適用は必須ではありません。** 画面の各機能が仕様通りに動作することが最優先であり、デザインの微調整や装飾は「追加機能」の範囲として扱います。

5.1 共通・検索機能

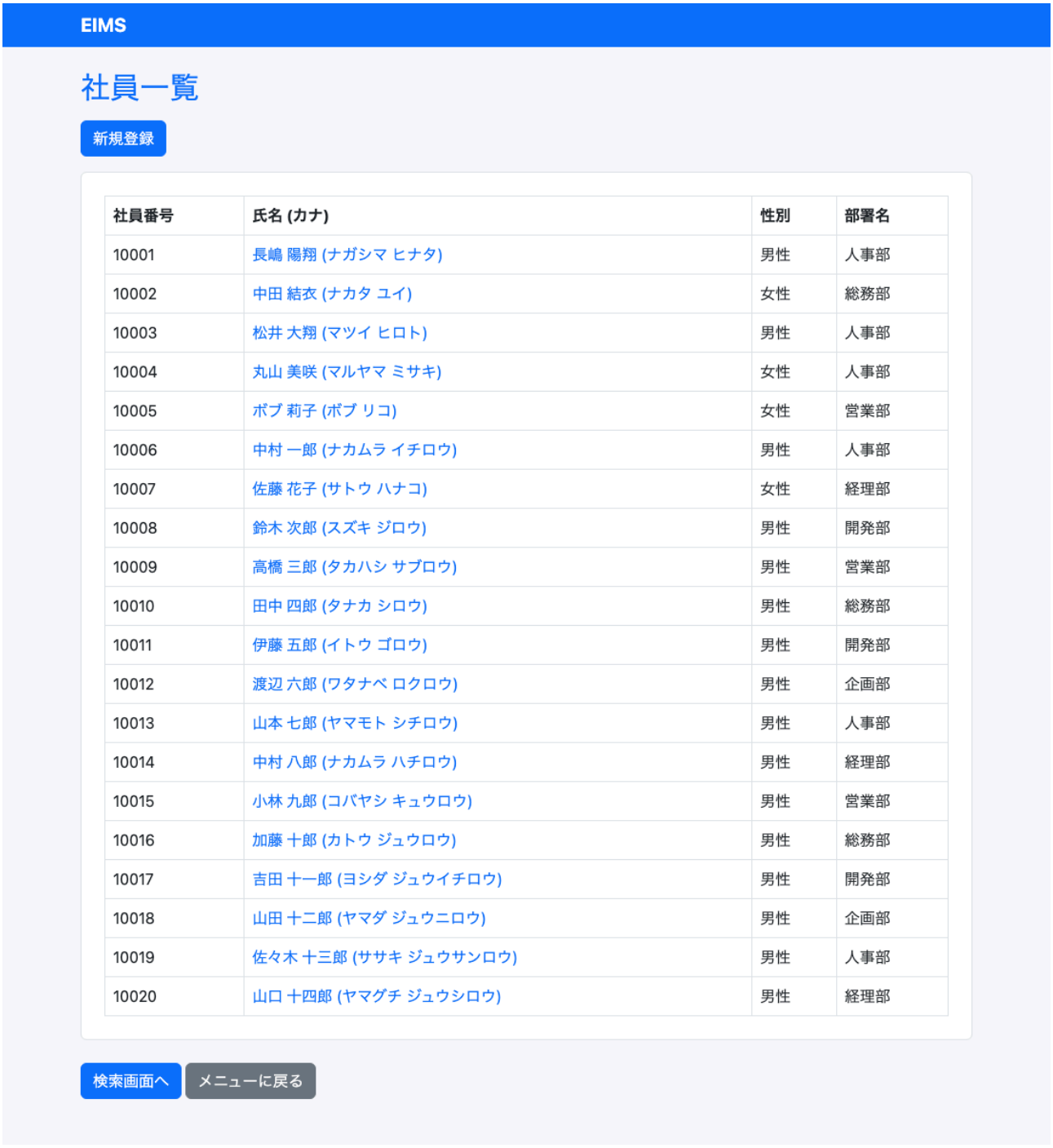
UI1: トップページ



トップページ

UI_L: 社員一覧画面

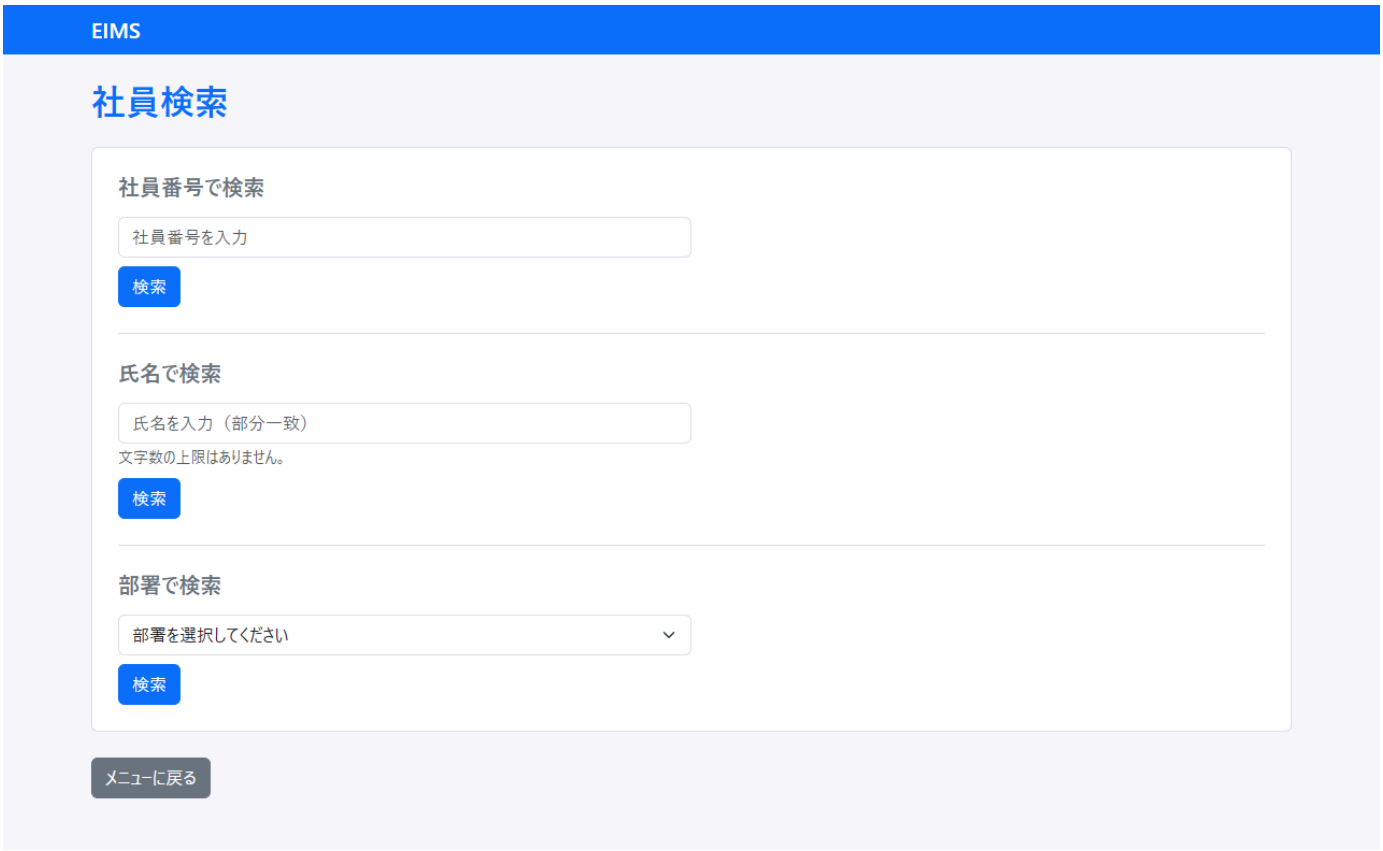
社員全員（初期データ20件）のリストが表示されます。



社員一覧画面

UI2: 検索条件入力画面

「社員番号」「氏名」「所属部署」の3つの検索方式を提供します。

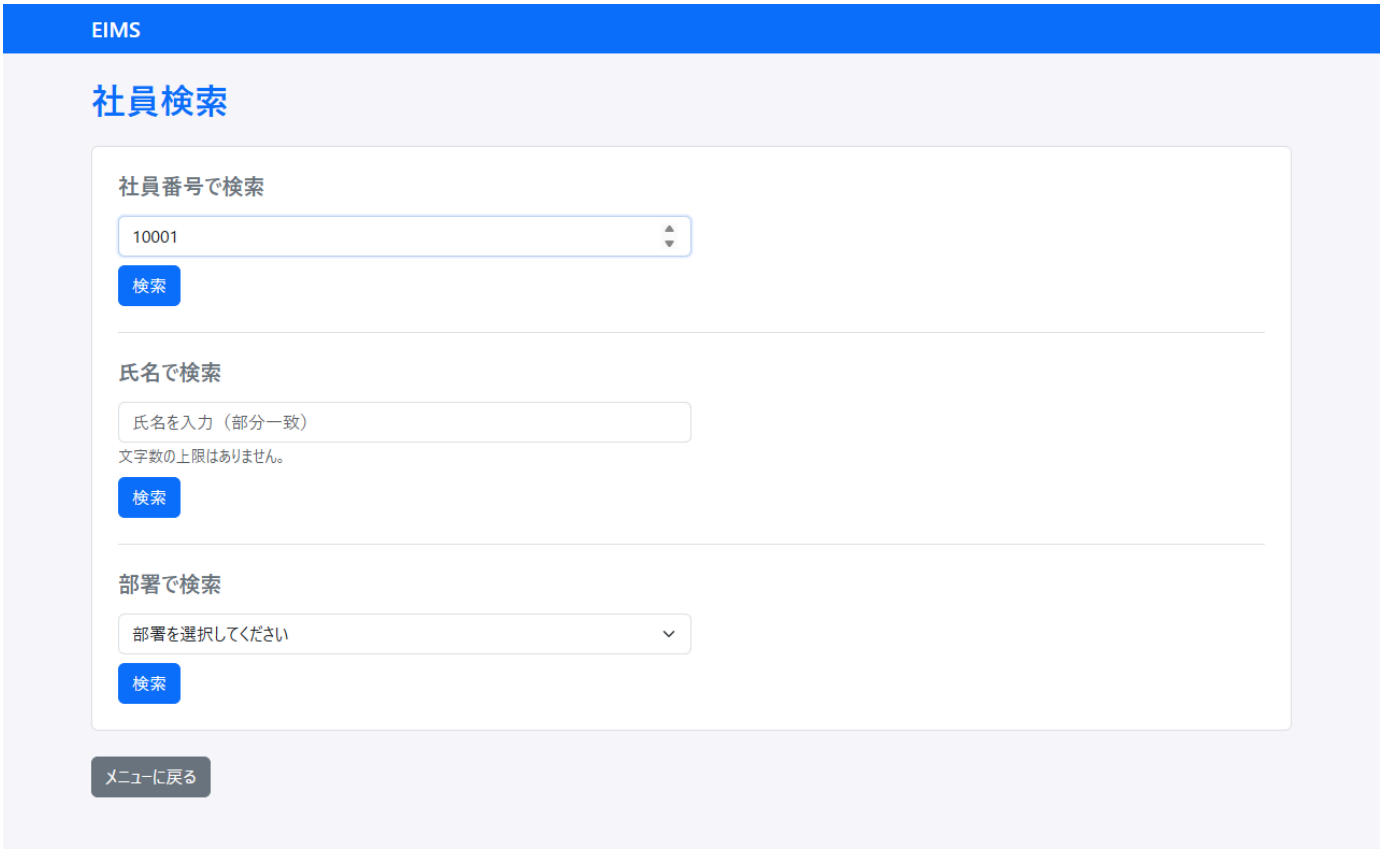


検索条件入力画面

UI2・UI3: 検索機能の実行例（Before / After）

検索機能では、入力した条件に応じて結果の表示先が異なります。それぞれの検索パターンにおける「検索条件入力時（Before）」と「検索結果（After）」の流れを以下に示します。

【パターン1】 社員番号による検索 社員番号 `10001` を指定して検索を実行します。社員番号は一意であるため、結果が1件に特定され、「**社員詳細画面**」**へ直接遷移**します。

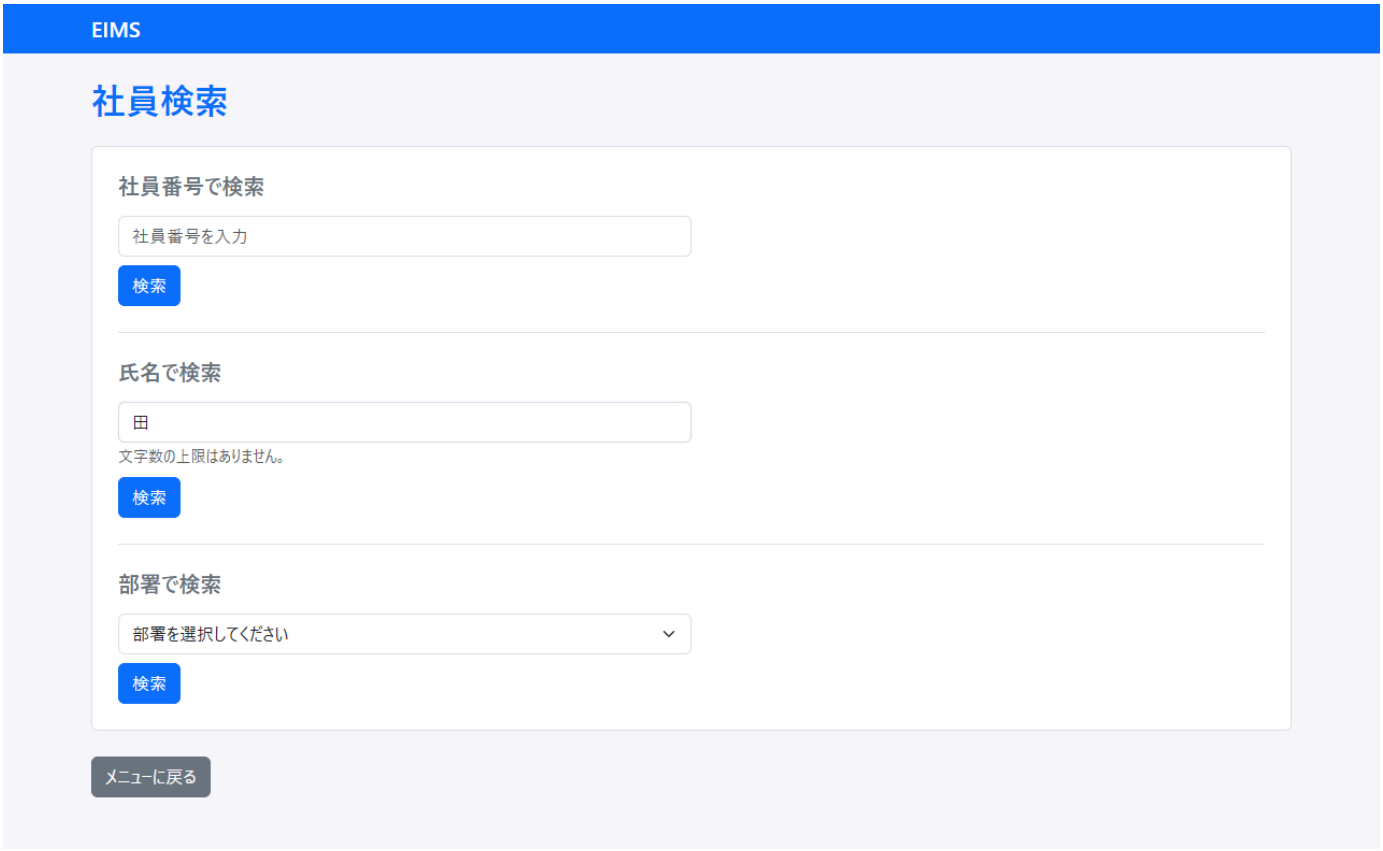


社員番号検索（入力時）



社員番号検索（結果：詳細画面へ遷移）

【パターン2】 氏名による検索 名前に 田 を含む社員を検索します。氏 または 名 にキーワードが含まれる社員が抽出されます。複数の該当者が存在する可能性があるため、「**検索結果一覧画面**」**へ遷移**し、ヒットした社員のリストを表示します。

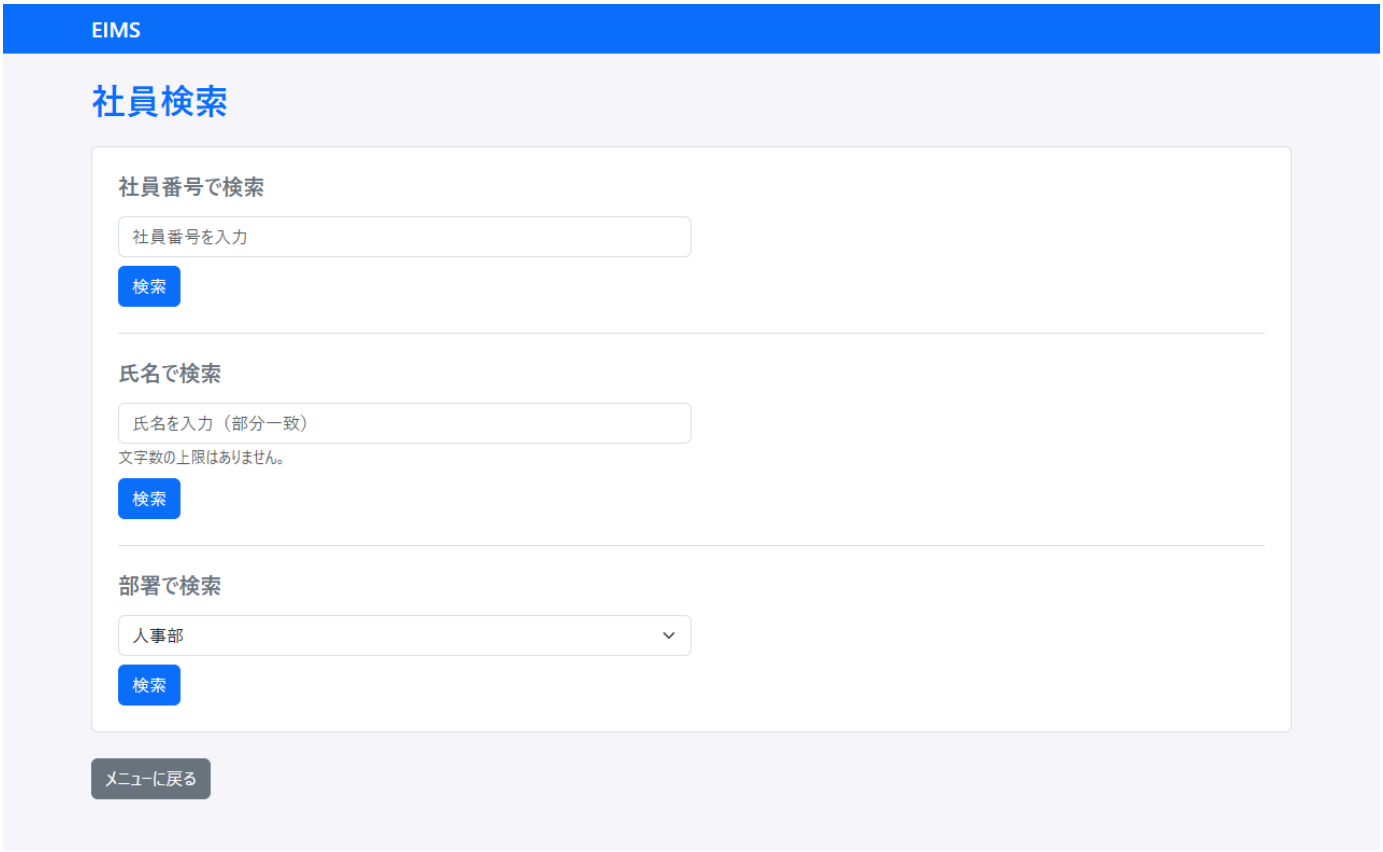


氏名検索（入力時）



氏名検索（結果：一覧表示）

【パターン3】 部署による検索 プルダウンから 人事部 を選択して検索を実行します。選択した部署に所属する全社員が抽出され、「**検索結果一覧画面**」**にリスト表示**されます。



所属部署検索（入力時）

EIMS

検索結果一覧

社員番号	氏名 (カナ)	性別	部署名
10001	長嶋 陽翔 (ナガシマ ヒナタ)	男性	人事部
10003	松井 大翔 (マツイ ヒロト)	男性	人事部
10004	丸山 美咲 (マルヤマ ミサキ)	女性	人事部
10006	中村 一郎 (ナカムラ イチロウ)	男性	人事部
10013	山本 七郎 (ヤマモト シチロウ)	男性	人事部
10019	佐々木 十三郎 (ササキ ジュウサンロウ)	男性	人事部

検索画面に戻る　メニューに戻る

所属部署検索（結果：一覧表示）

UI_D: 社員詳細画面



社員詳細画面

5.2 登録機能

UI4: 社員情報登録画面

新規に社員情報（山田太郎さん）を入力した状態の画面です。社員番号は自動採番されるため、入力項目にはありません。



社員情報登録画面

UI5: 登録確認画面

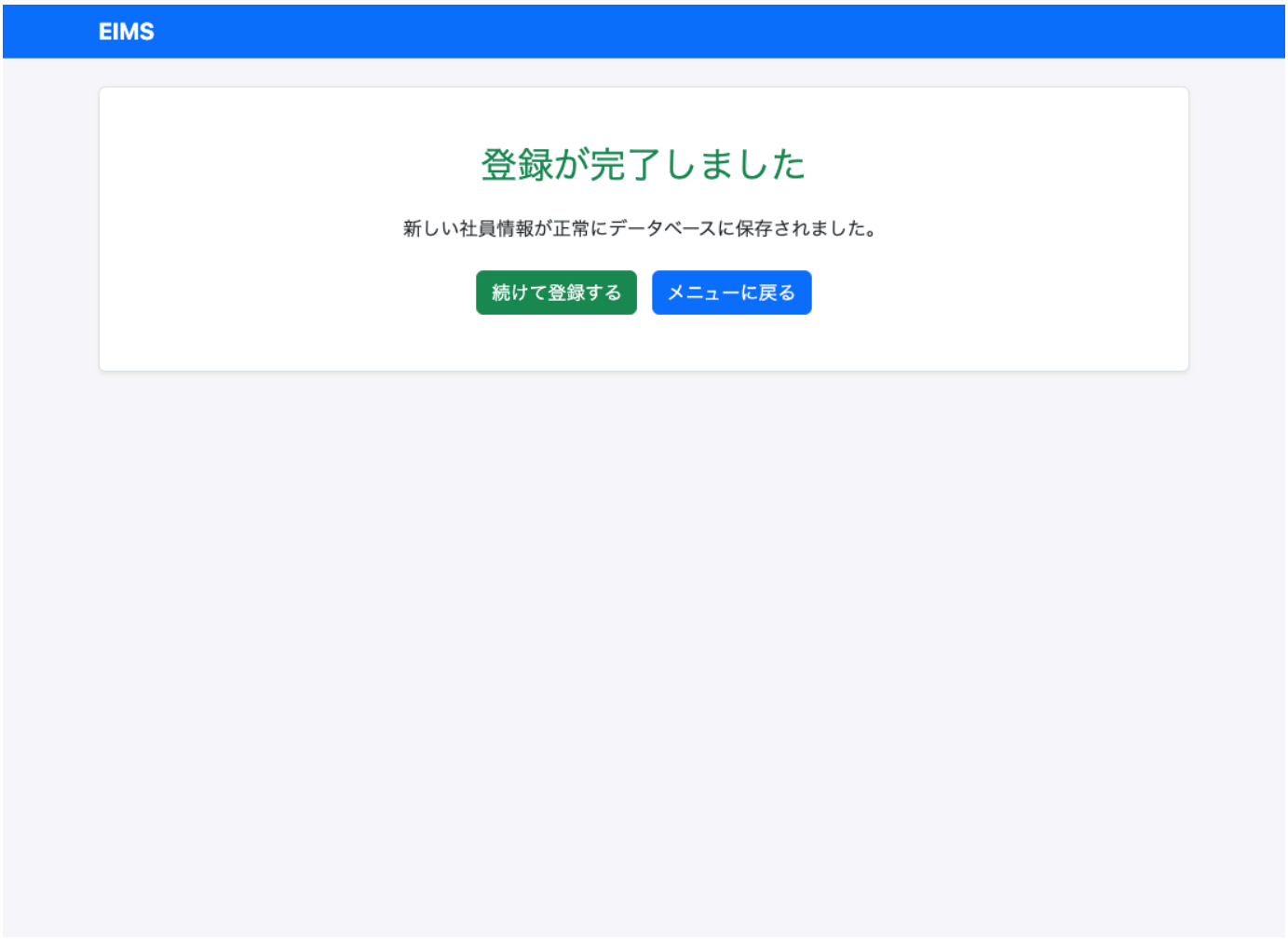
入力した内容に間違いがないかを確認する画面です。



登録確認画面

UI6: 登録完了画面

登録処理が成功したことを通知する画面です。



登録完了画面

5.3 変更機能

UI9: 変更画面

対象社員（山田 太郎さん）の情報を変更します。ここでは例として名を「次郎」に修正しています。



変更画面

UI10: 変更確認画面

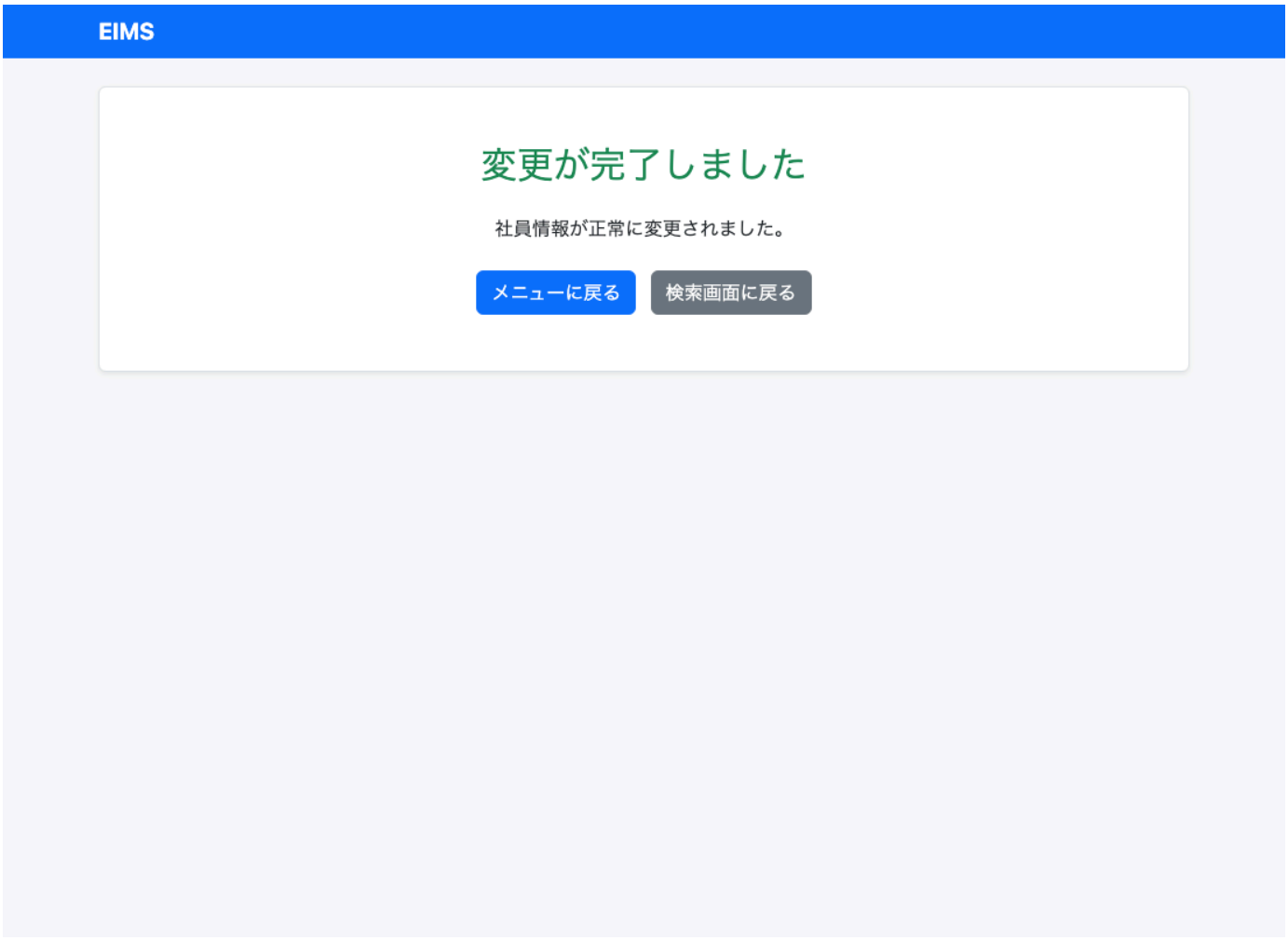
修正した内容（変更後の名など）を確定前に確認する画面です。



変更確認画面

UI11: 変更完了画面

情報の変更処理が成功したことを通知する画面です。



変更完了画面

5.4 削除機能

UI7: 削除確認画面

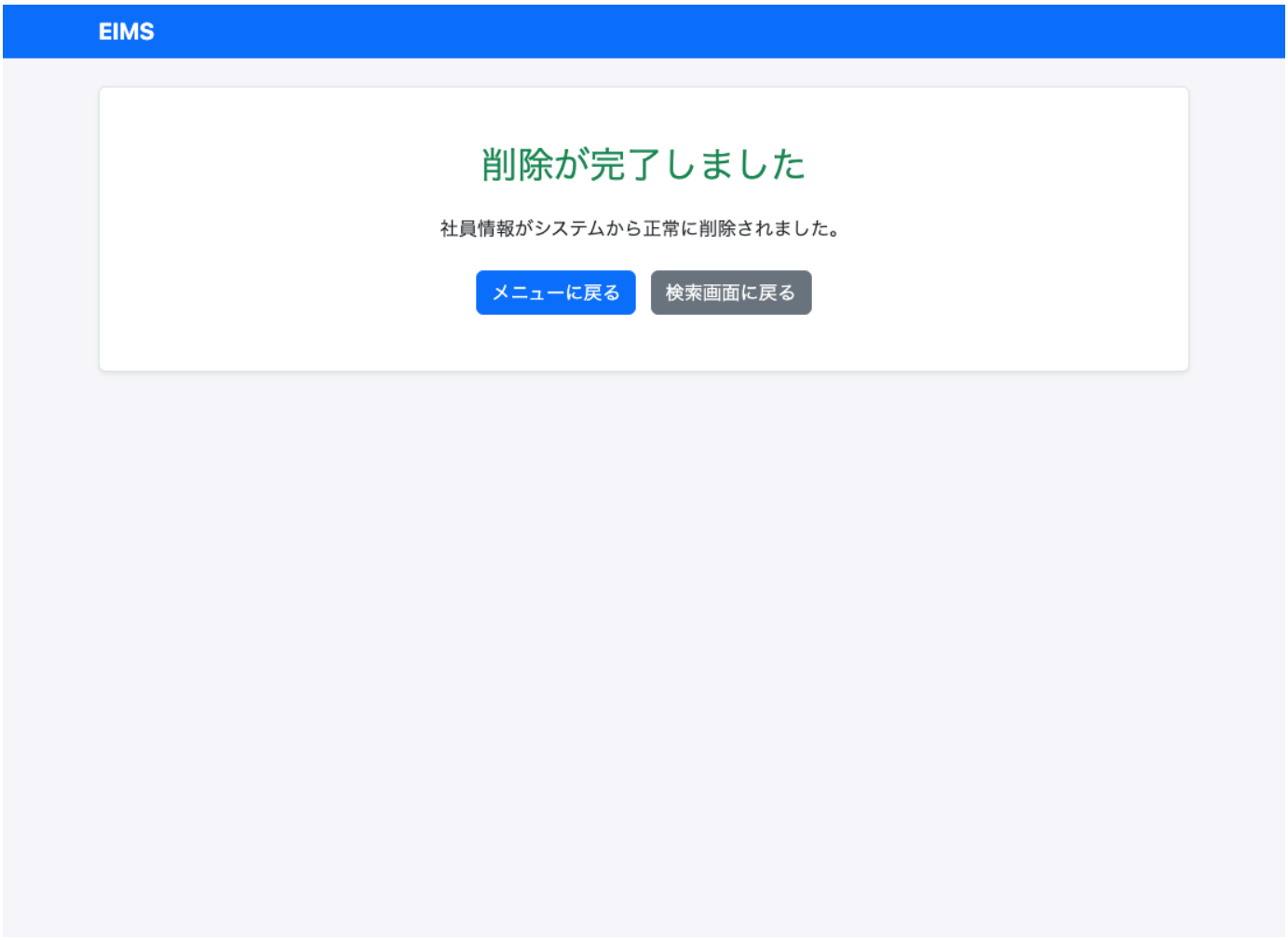
対象社員（山田 次郎さん）をデータベースから本当に削除してよいかを確認する画面です。



削除確認画面

UI8: 削除完了画面

削除処理が成功したことを通知する画面です。



削除完了画面

5.5 入力チェック仕様

5.5.1 検索

検索フォームごとに、対応するControllerメソッドで次の条件を確認する。入力エラー時は `message` と部署一覧 `departments` をModelに設定して `search` を返す。検索画面は `message` がある場合に文言を表示する。入力エラーではEmployeeServiceの検索メソッドを呼び出さない。

検索条件	入力条件	入力エラー時のメッセージ
社員番号 `empNo`	必須。未送信・空文字・空白のみ（半角・全角）で `Integer` の値が得られない場合は未入力として扱う	V013：社員番号を入力してください。
社員名 `keyword`	必須、文字数上限なし。`null`・空文字・空白のみ（半角・全角スペース等）は未入力として扱う。前後の空白は保持する	V014：社員名を入力してください。
部署 `deptNo`	プルダウンで選択必須。未選択・未送信・空文字・空白のみ（半角・全角）は未選択として扱う	V015：部署を選択してください。

- 社員名は未入力のみを判定し、有効なキーワードは長さにかかわらずそのまま検索へ渡す。前後空白は除去せず、検索条件に含める。
- 社員名入力欄には `maxlength` を設定せず、サーバー側でもキーワードの長さを理由に検索を拒否しない。検索キーワードはDBに保存しないため、登録・変更時の氏・名の10文字制限およびDB列の `VARCHAR(10)` とは区別する。10文字を超える入力も通常どおり検索し、該当社員がいなければ0件を表示する。
- 社員番号は5桁固定ではない。`Integer` の範囲（-2147483648〜2147483647）で受け取り、0・負数も通常検索と同じく該当社員がなければ0件として扱う。整数に変換できない文字列や範囲外の値は従来どおりHTTP 400となり、独自エラー画面は追加しない。
- プルダウン値の改ざんなど、2.1節でテスト対象外とした操作の扱いは変更しない。
- **未入力と検索結果0件は区別する。** 有効な条件で検索を実行し、該当社員がない場合は `search_result` にE001を表示する。

5.5.2 登録・変更

「社員登録」および「社員変更」において実施する入力チェックルールを以下に定義する。

項目名	必須	文字数制限	備考
氏（漢字）	○	10文字以内	`lastName`
名（漢字）	○	10文字以内	`firstName`
氏（カナ）	○	20文字以内	`lastKana`
名（カナ）	○	20文字以内	`firstKana`
パスワード	○	4〜16文字	`password`
性別	○	-	ラジオボタン（1:男, 2:女）
部署	○	-	プルダウン選択

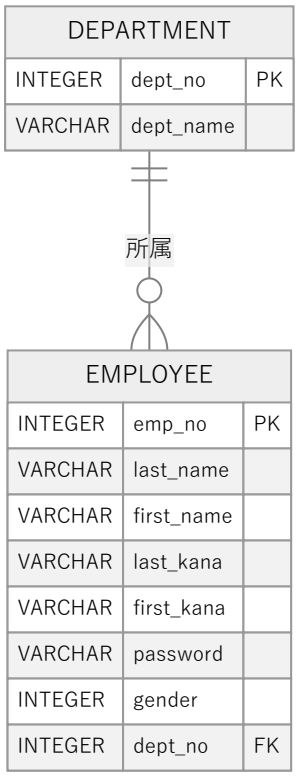
> ※ **パスワードの取り扱いについて**：本演習では、入力チェック（必須・文字数）の学習に集中するため、パスワードは平文（ハッシュ化なし）のまま保存する。ただし、登録確認画面と変更確認画面では、1.2.2節・1.3.2節のとおりマスク表示する。**実務では、パスワードは必ずハッシュ化（BCrypt など）して保存**し、平文のまま保存・出力してはならない。本演習の実装はあくまで学習用であり、そのまま実運用に用いないこと。

6. データベース仕様

6.1 データベース概要

項目	内容
データベース名	eimsdb
DBMS	MySQL
文字コード	utf8mb4
照合順序	utf8mb4_unicode_ci

6.2 テーブルの関係性（実体関連図）



6.3 テーブル定義

`Field` はデータベースのテーブルの列名であり、Javaのフィールド名ではない。 DBでは `emp_no`、Javaでは `empNo` のように使い分ける。DBの列名をJavaへそのまま転記せず、Entityの属性名は9章のクラス図に従う。

本節のER図とテーブル定義は、共通機能で使用する列を示す。配布SQLには追加機能用の `role` と `delete_flg` もあらかじめ定義されている。これらは共通機能のEntityには不要であり、追加機能を実装する際に対応するJavaフィールドを追加する。列とフィールドの対応は追加機能設計書の3.6節・4.6節を参照する。

【部署情報テーブル】（department）

Field（テーブルの列名）	Type	Null	Key	Default	説明
dept_no	INTEGER	NOT NULL	PRIMARY KEY	なし	部署コード
dept_name	VARCHAR(10)	NOT NULL		なし	部署名

【社員情報テーブル】（employee）

Field（テーブルの列名）	Type	Null	Key	Default	説明
emp_no	INTEGER	NOT NULL	PRIMARY KEY	AUTO_INCREMENT	社員番号（自動採番）
last_name	VARCHAR(10)	NOT NULL		なし	氏
first_name	VARCHAR(10)	NOT NULL		なし	名
last_kana	VARCHAR(20)	NOT NULL		なし	氏（カナ）
first_kana	VARCHAR(20)	NOT NULL		なし	名（カナ）
password	VARCHAR(20)	NOT NULL		なし	パスワード
gender	INTEGER	NOT NULL		なし	性別（1:男性 2:女性）
dept_no	INTEGER	NOT NULL	FOREIGN KEY	なし	部署コード

外部キー制約:

- 制約名: `fk_employee_department`
- employee.dept_no → department.dept_no を参照

DB列名とJavaフィールド名の対応

DBの列名	Javaのフィールド名	Entity
`emp_no`	`empNo`	Employee
`last_name`	`lastName`	Employee
`first_name`	`firstName`	Employee
`last_kana`	`lastKana`	Employee
`first_kana`	`firstKana`	Employee
`password`	`password`	Employee
`gender`	`gender`	Employee
`dept_no`	`deptNo`	Employee・Department
`dept_name`	`deptName`	Department

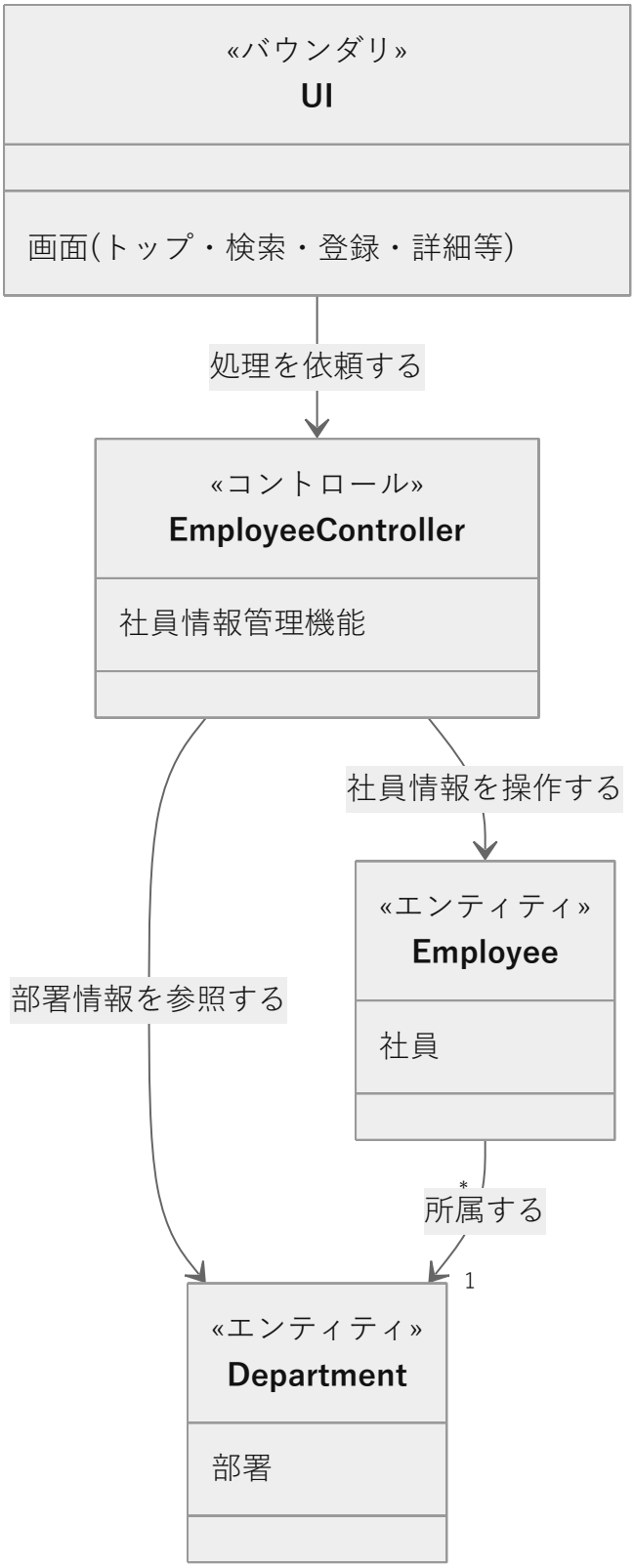
例えば、Employeeの `empNo` には `@Column(name = "emp_no")` を付けてDB列に対応付ける。`department` と `employees` はEntity間の関連を表すJavaフィールドであり、同名のDB列は作らない。

6.4 初期データ仕様

テーブル	件数	備考
department	6件	人事部(100)、経理部(200)、営業部(300)、総務部(400)、開発部(500)、企画部(600)
employee	20件	テスト用サンプルデータ（社員番号 10001〜10020）

7. クラス図（分析レベル）

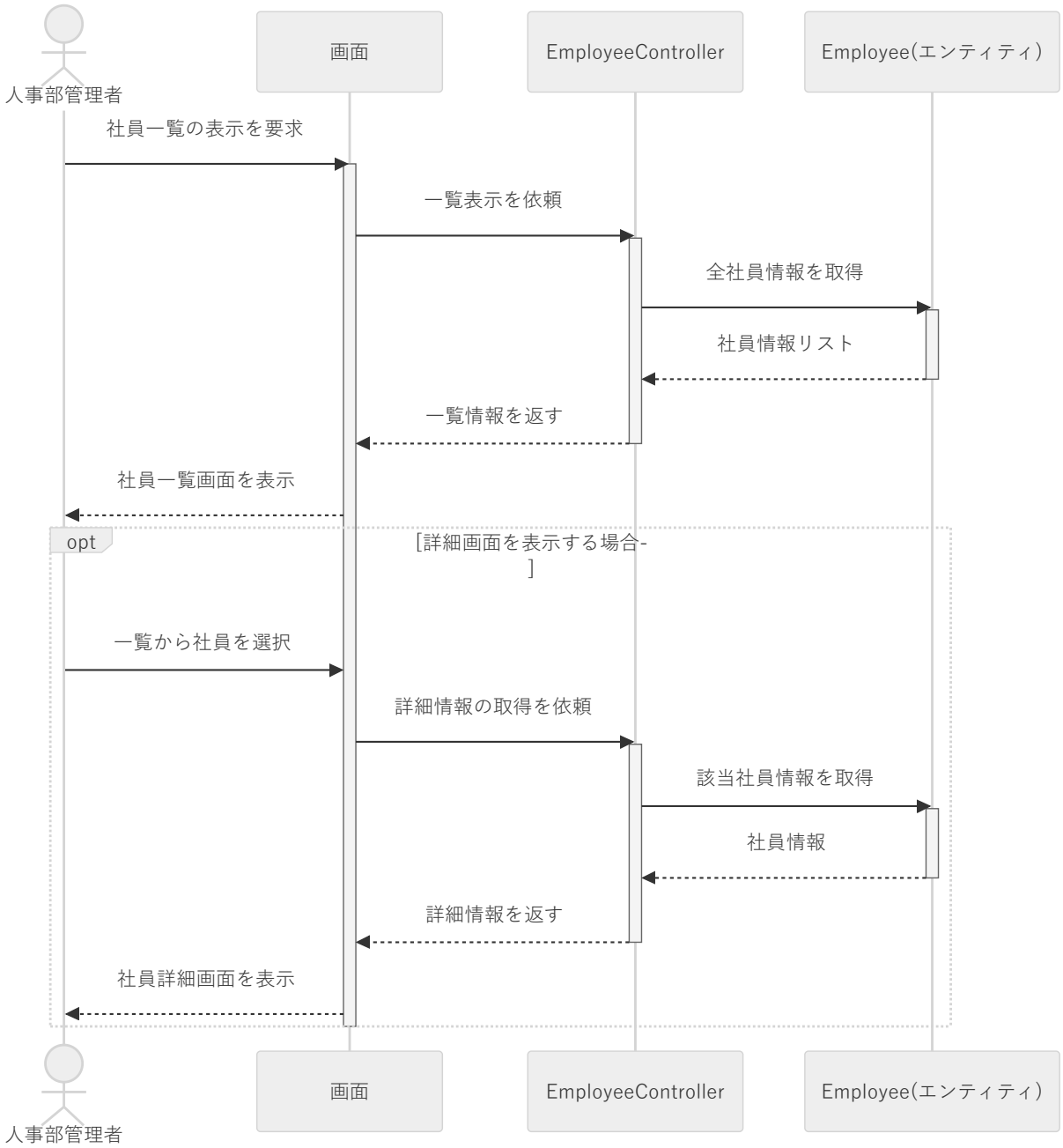
システムに登場する概念を「バウンダリ（画面）」「コントロール（制御・管理）」「エンティティ（データ）」の役割（BCEモデル）で定義する。



8. シーケンス図（分析レベル）

各ユースケースが実現されるまでのオブジェクト間のメッセージのやり取り（処理の流れ）を以下に定義する。

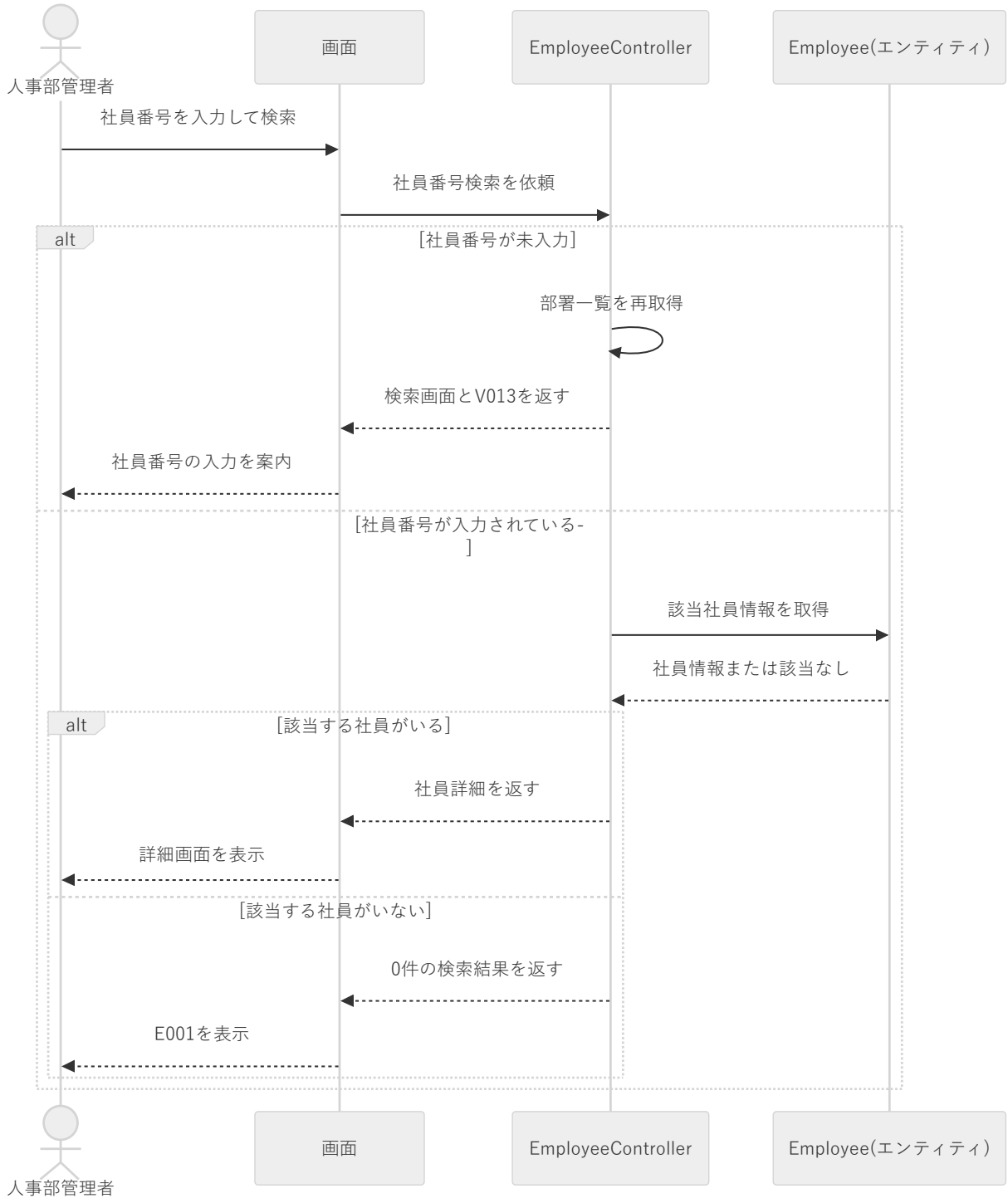
8.1 UC001: 社員一覧を表示する



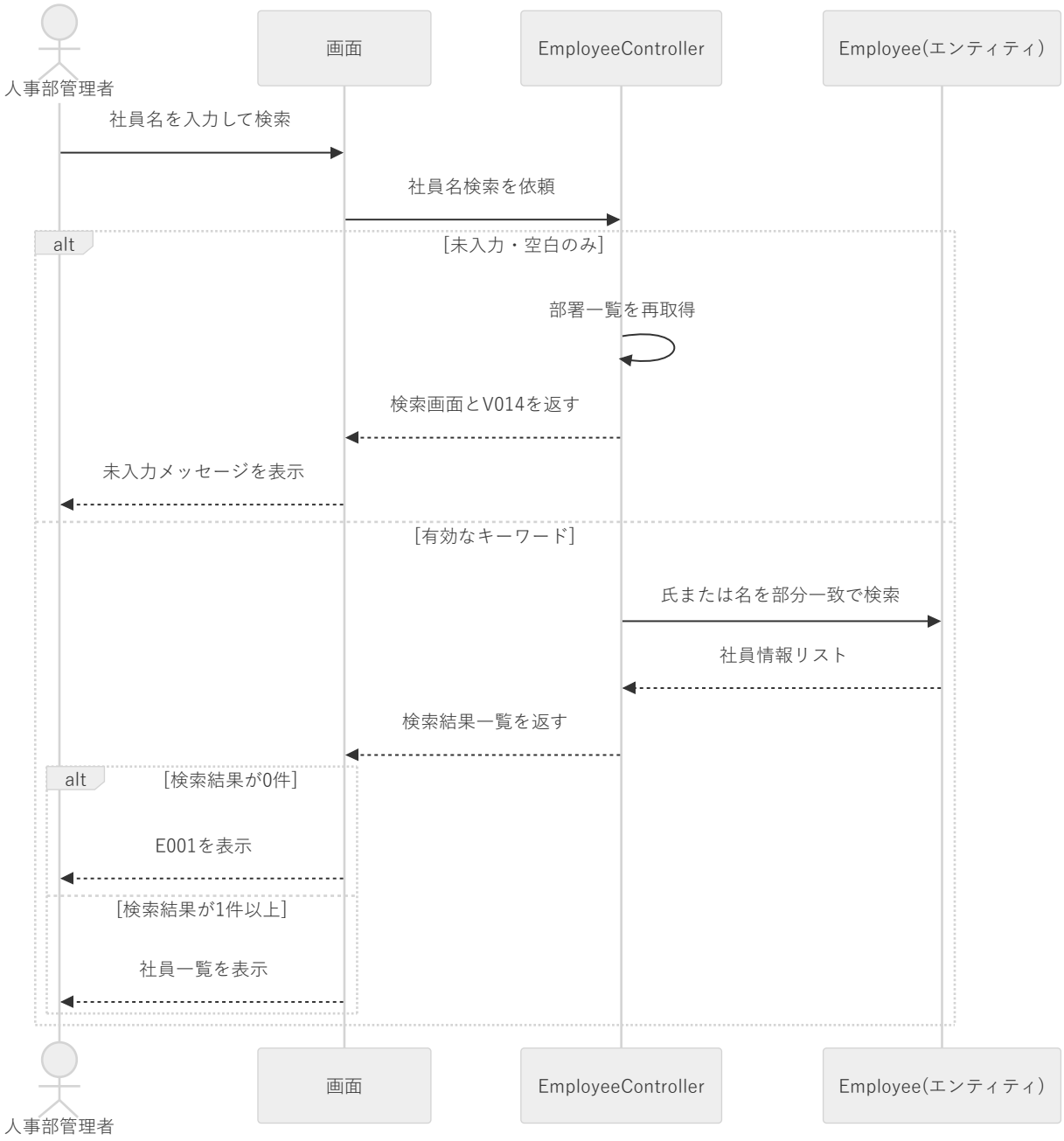
8.2 UC002: 社員情報を検索する

初回表示では、プルダウン用の部署一覧を取得して検索画面を表示する。以下は、検索画面から各条件を送信した後の流れである。入力エラー時は社員を検索せず、部署一覧を再取得して検索画面にメッセージを表示する。

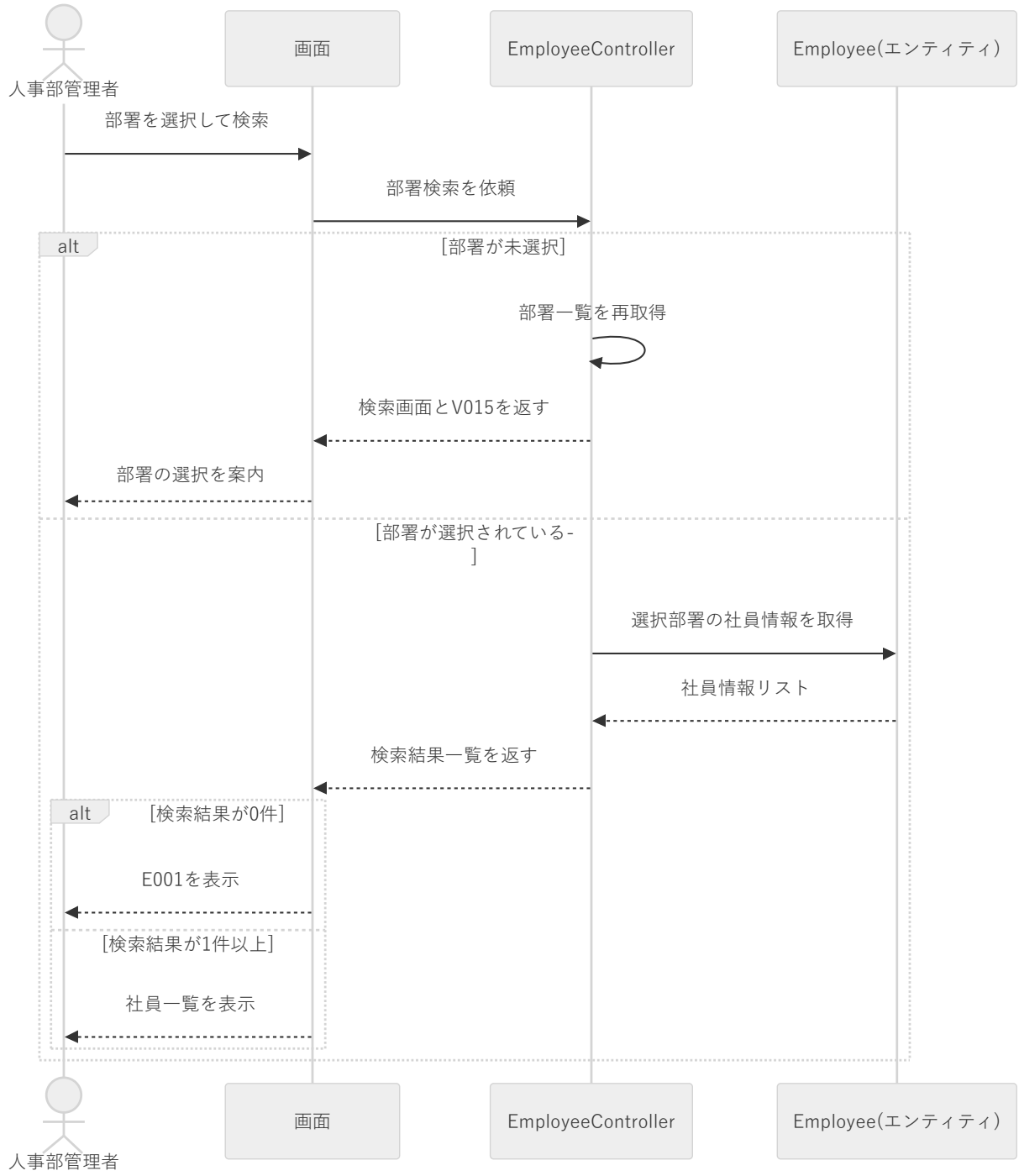
社員番号で検索する


人事部管理者
画面
EmployeeController
Employee(エンティティ)
社員番号を入力して検索
社員番号検索を依頼
alt
[社員番号が未入力]
部署一覧を再取得
検索画面とV013を返す
社員番号の入力を案内
[社員番号が入力されている-
]
該当社員情報を取得
社員情報または該当なし
alt
[該当する社員がいる]
社員詳細を返す
詳細画面を表示
[該当する社員がいない]
0件の検索結果を返す
E001を表示
人事部管理者
画面
EmployeeController
Employee(エンティティ)

社員名で検索する

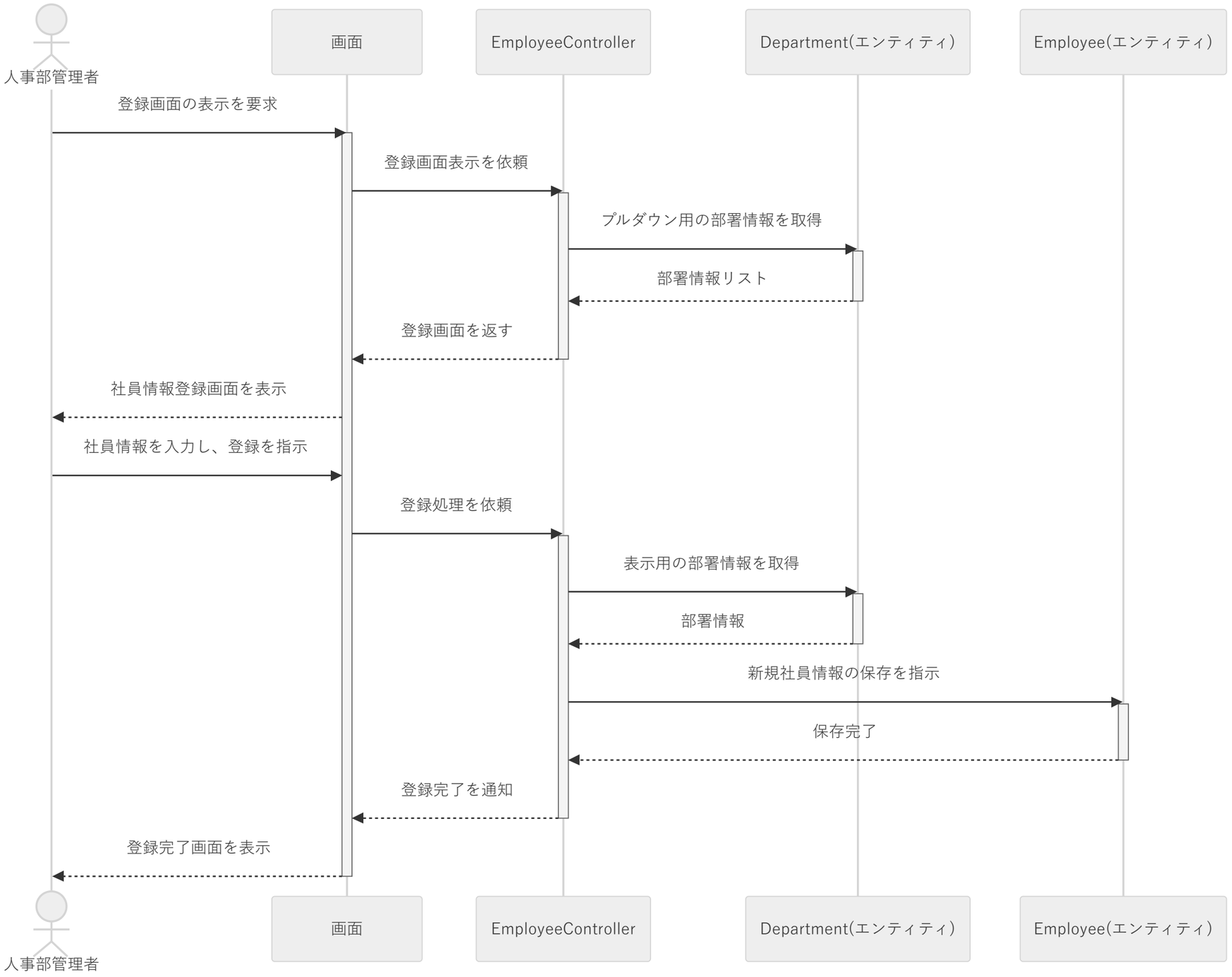

人事部管理者
画面
EmployeeController
Employee(エンティティ)
社員名を入力して検索
社員名検索を依頼
alt
[未入力・空白のみ]
部署一覧を再取得
検索画面とV014を返す
未入力メッセージを表示
[有効なキーワード]
氏または名を部分一致で検索
社員情報リスト
検索結果一覧を返す
alt
[検索結果が0件]
E001を表示
[検索結果が1件以上]
社員一覧を表示
人事部管理者
画面
EmployeeController
Employee(エンティティ)

部署で検索する

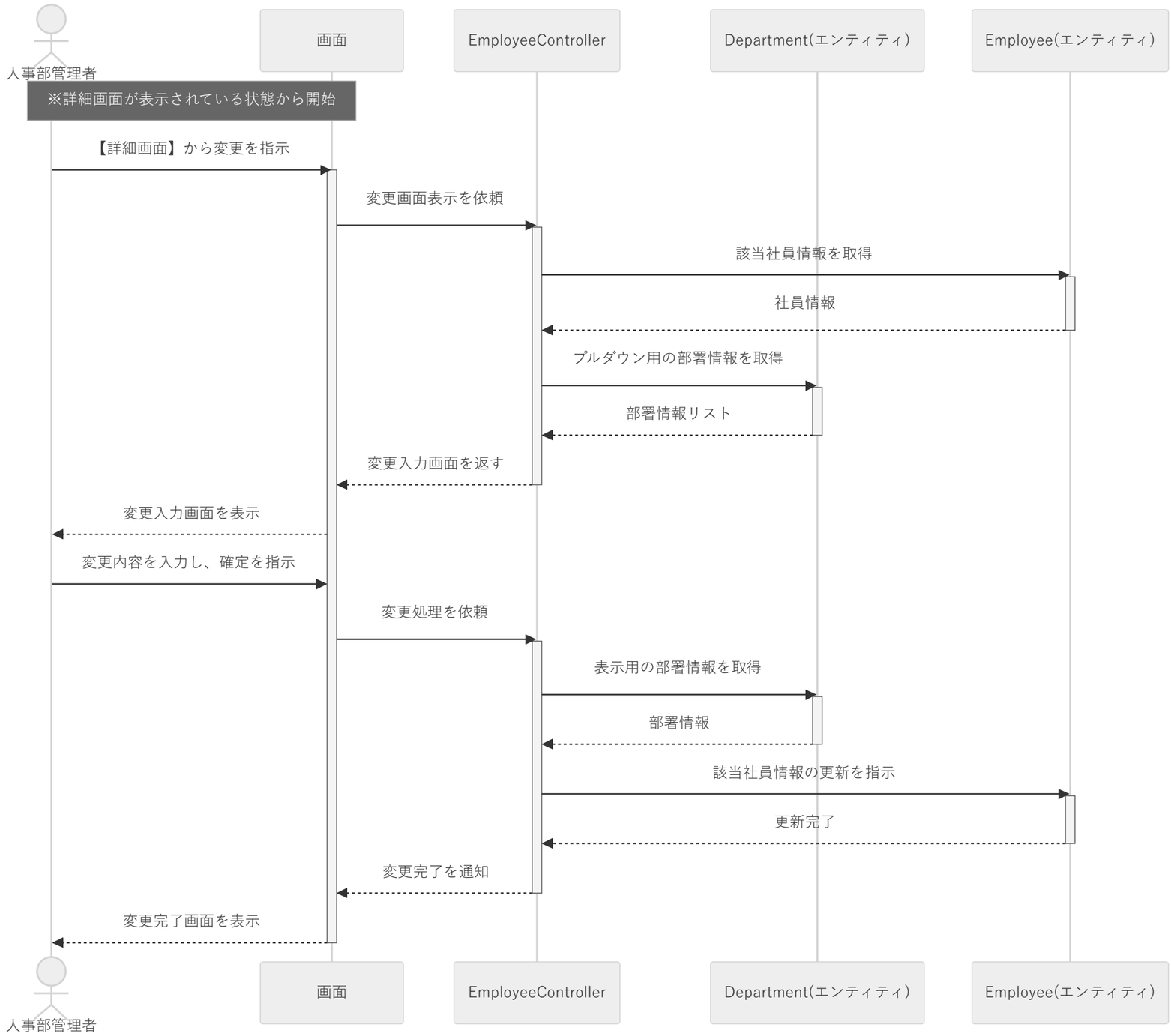


検索結果一覧の氏名を選択した場合は、対象社員の情報を取得して社員詳細画面を表示する。各メッセージの文言は12章を参照する。

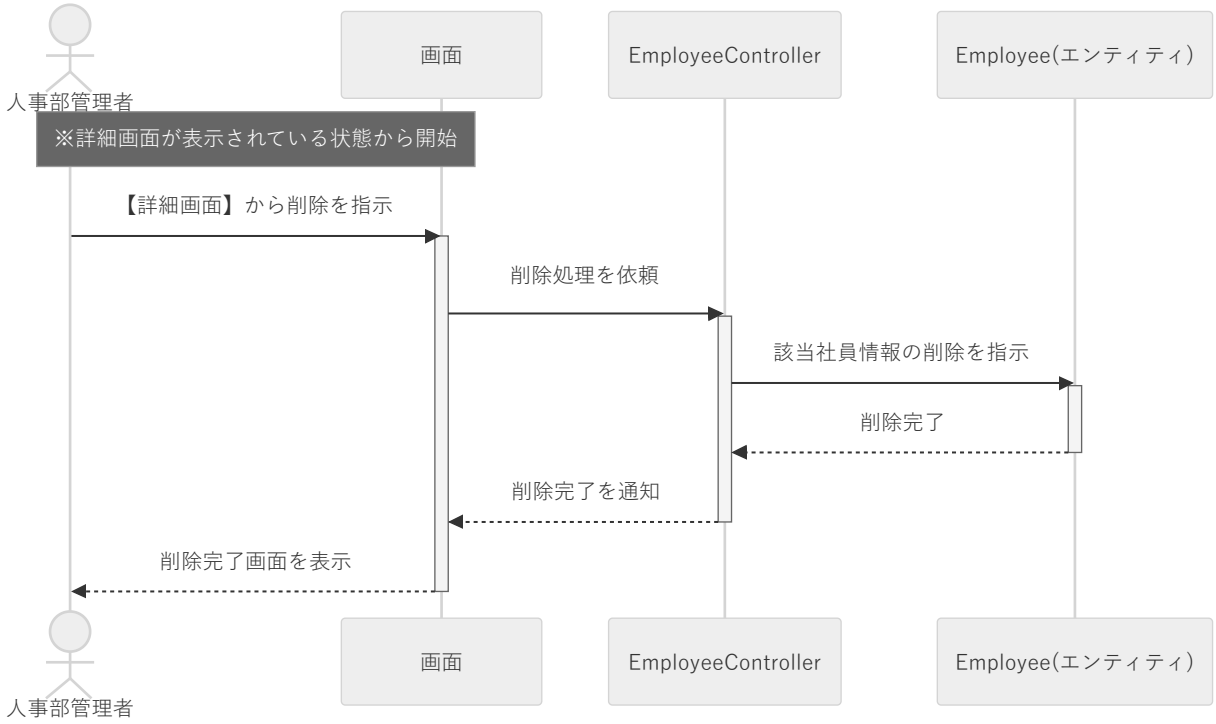
8.3 UC003: 社員情報を登録する



8.4 UC004: 社員情報を変更する



8.5 UC005: 社員情報を削除する

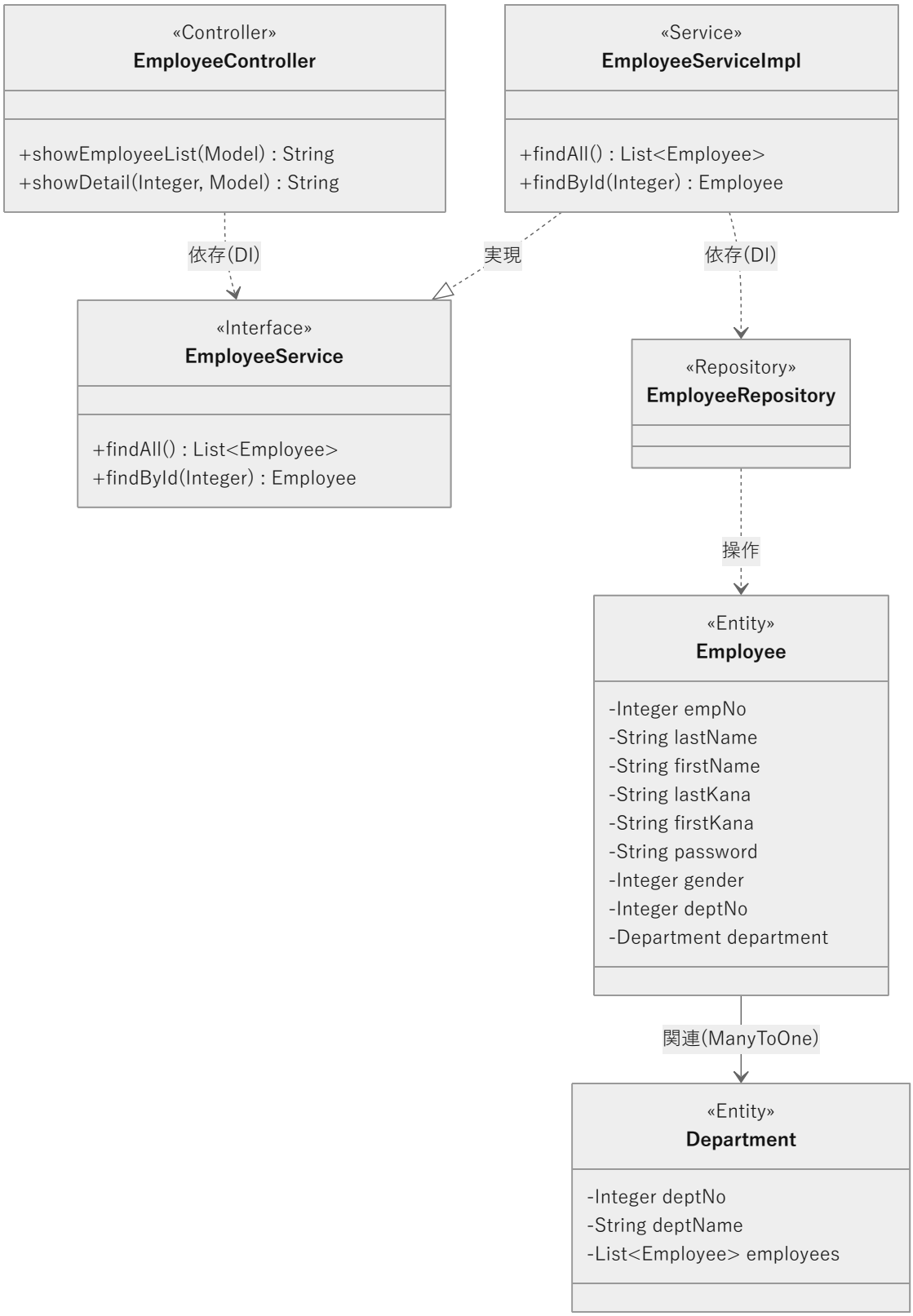


9. クラス図（設計レベル）

Spring Bootのアーキテクチャに基づき、各ユースケースの実装に必要なクラスとその依存関係を機能単位で定義する。

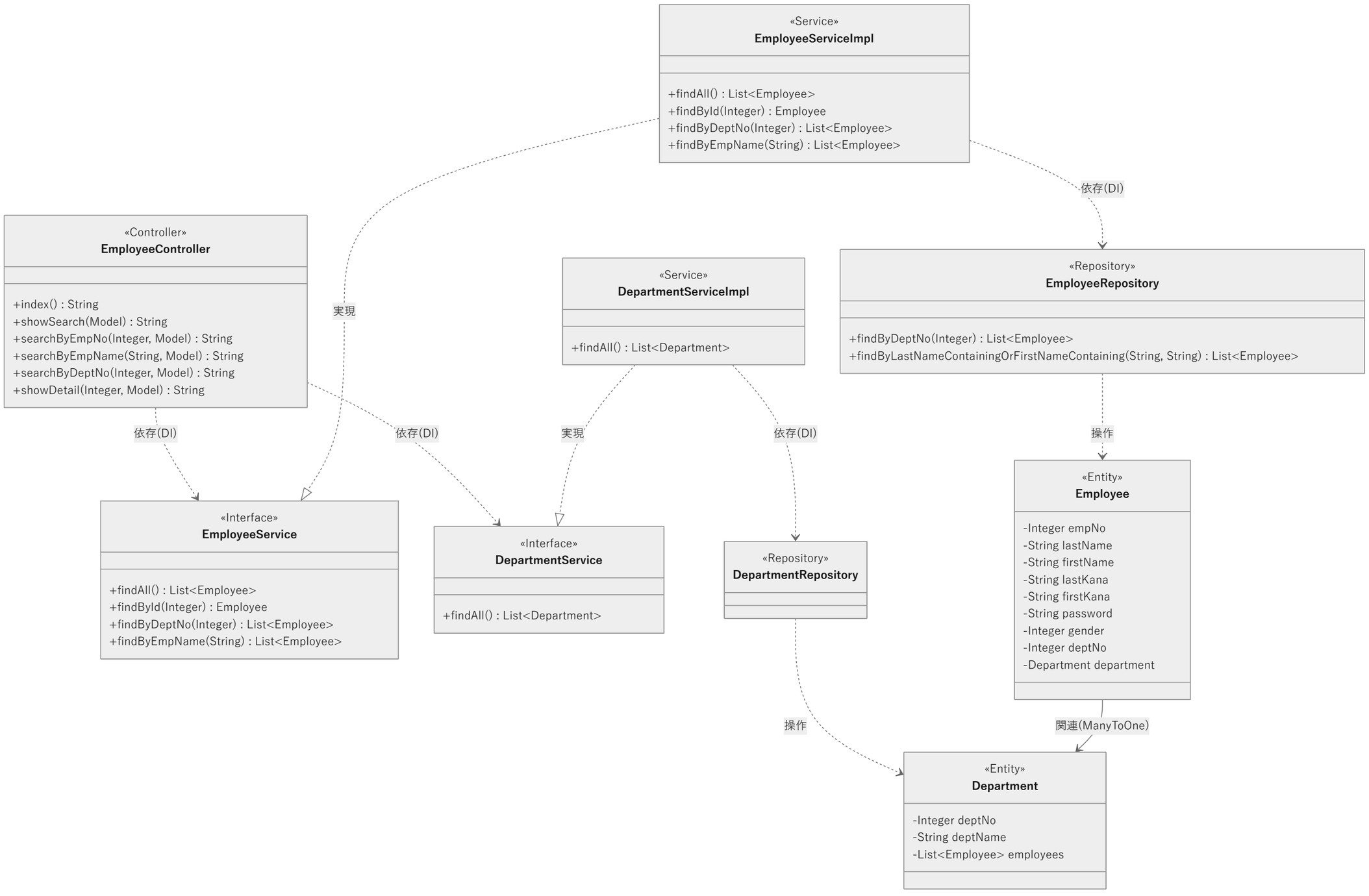
9.1 UC001: 社員一覧を表示する

一覧表示機能におけるクラス構成を以下に定義する。



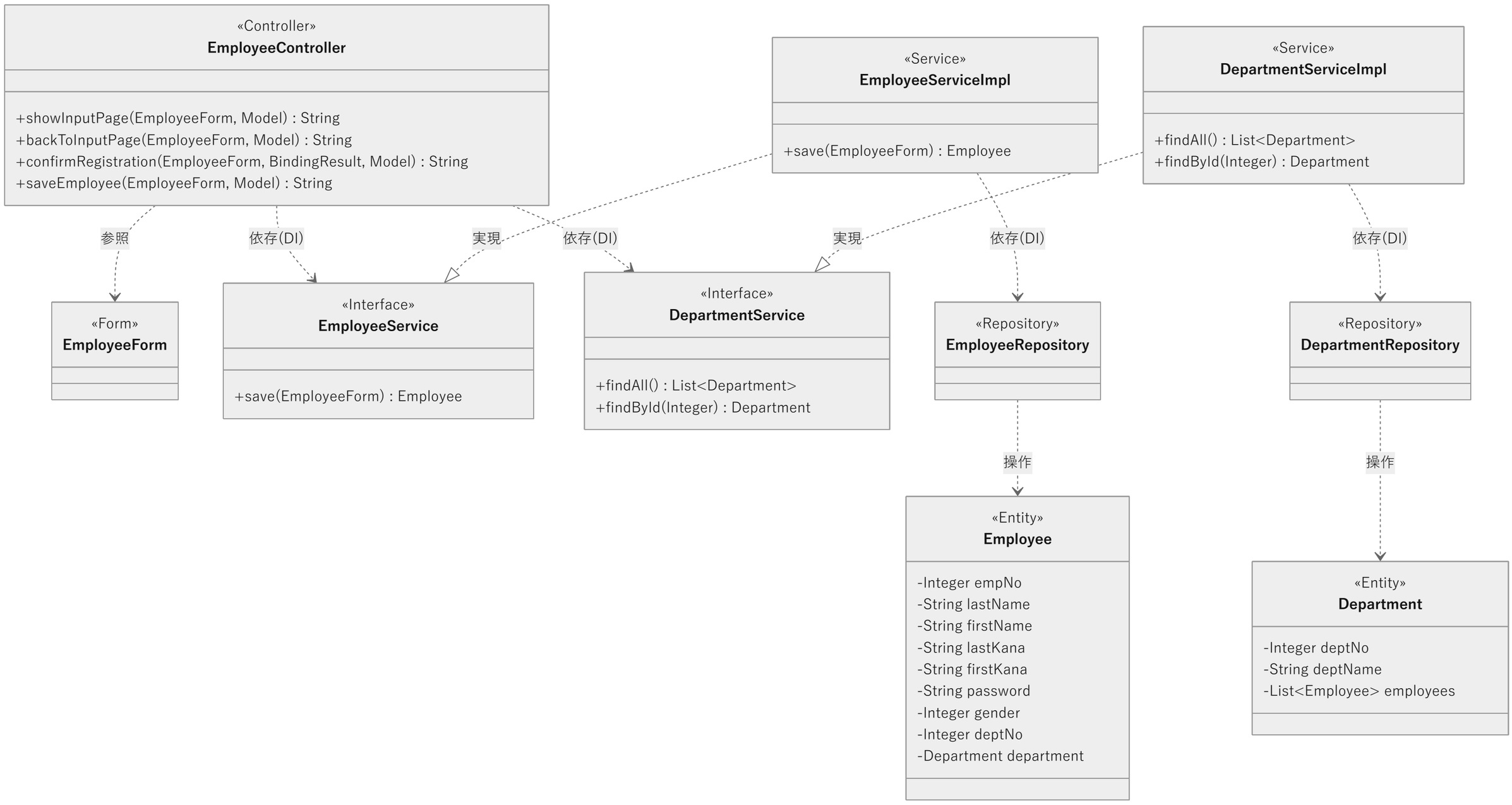
9.2 UC002: 社員情報を検索する

検索機能におけるクラス構成を以下に定義する。



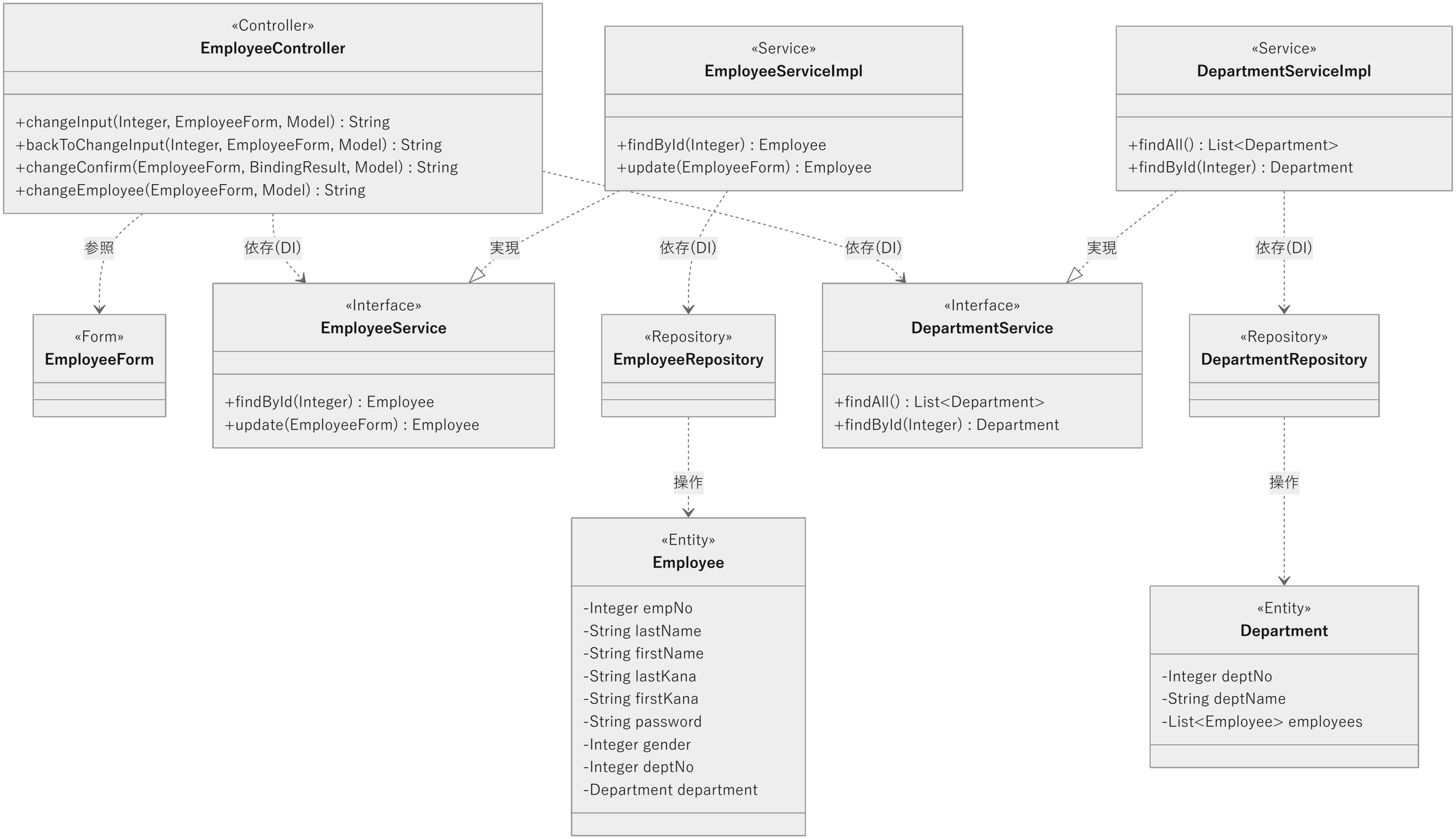
9.3 UC003: 社員情報を登録する

登録機能におけるクラス構成を以下に定義する。



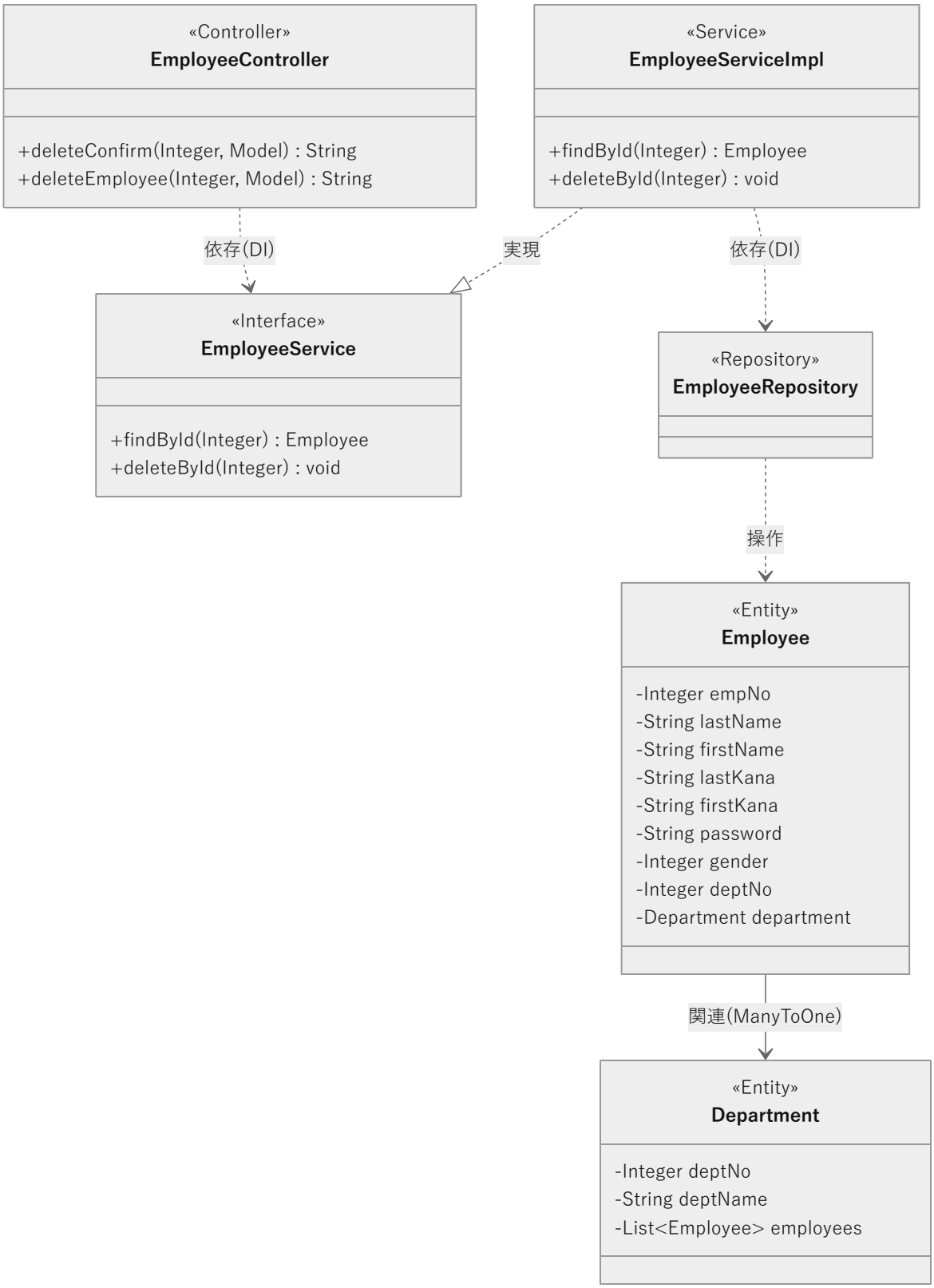
9.4 UC004: 社員情報を変更する

変更機能におけるクラス構成を以下に定義する。



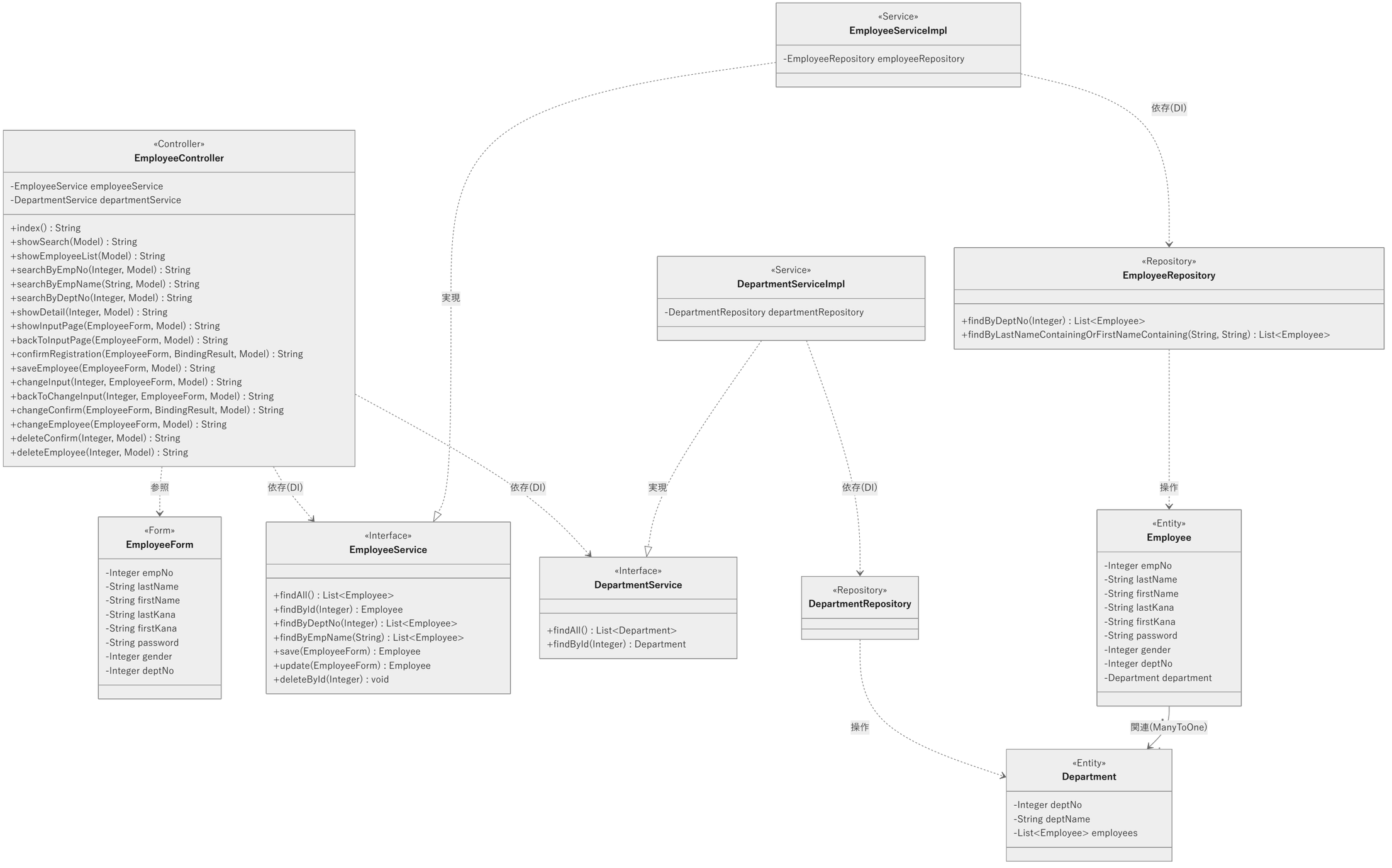
9.5 UC005: 社員情報を削除する

削除機能におけるクラス構成を以下に定義する。



9.6 全体クラス図

システム全体の網羅的なクラス構成および依存関係を以下に定義する。



10. シーケンス図（設計レベル）

10.1 URLマッピング（インターフェース仕様）一覧

フロントエンド（画面）からバックエンド（EmployeeController）へのリクエストとなる、エンドポイント（URL）と処理の対応関係を以下に定義する。

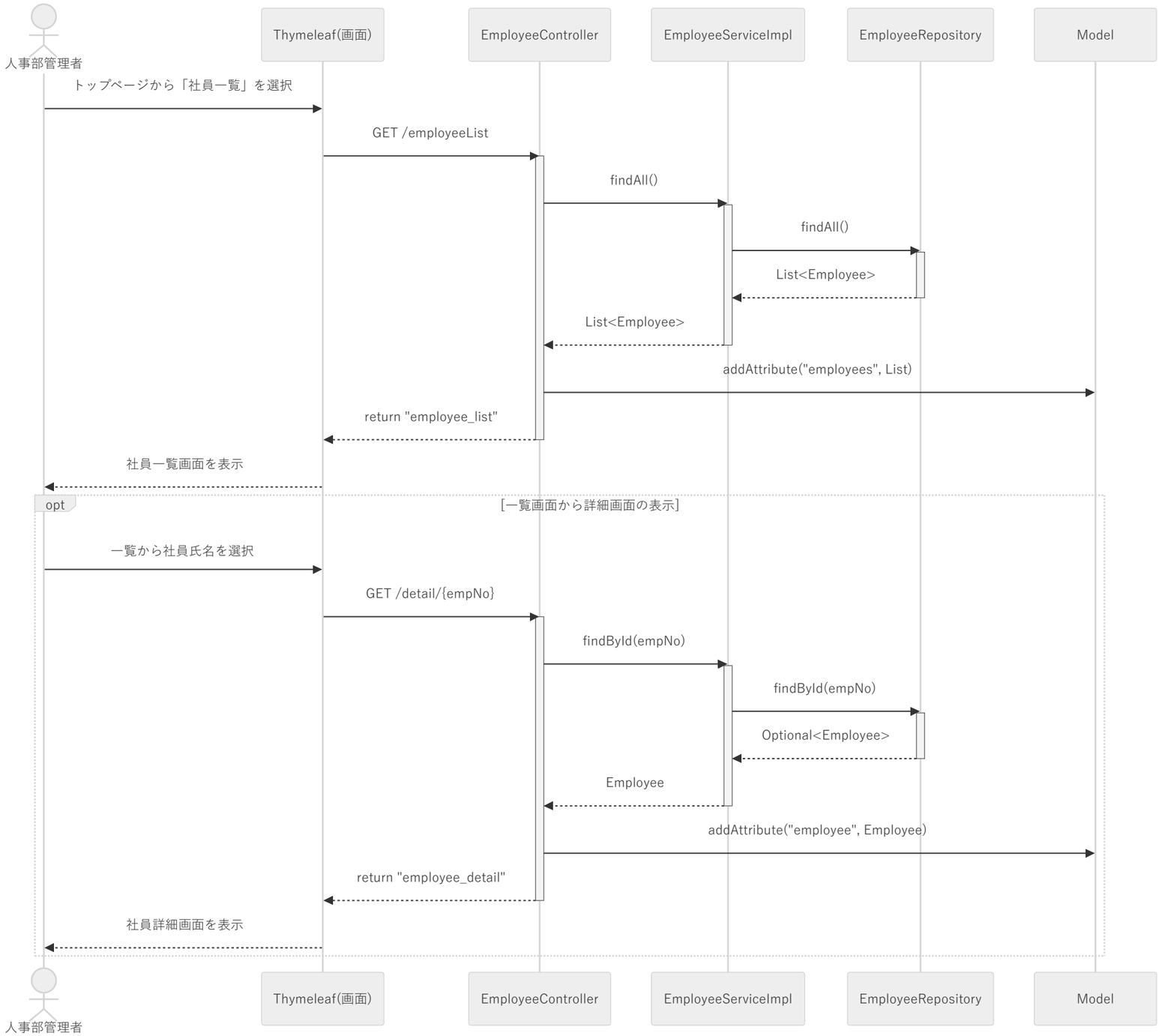
ユースケース	機能・アクション	HTTP	URLパス	コントローラーメソッド	遷移先ビュー名
共通	トップページ表示	GET	/, /index	index	index
UC001	社員一覧表示	GET	/employeeList	showEmployeeList	employee_list
UC001・002	社員詳細画面表示	GET	/detail/{empNo}	showDetail	employee_detail
UC002	検索画面表示	GET	/search	showSearch	search
UC002	氏名で検索	GET	/selectByEmpName	searchByEmpName	search_result／search（未入力時）
UC002	部署で検索	GET	/selectByDeptNo	searchByDeptNo	search_result／search（未入力時）
UC002	社員番号で検索	GET	/selectByEmpNo	searchByEmpNo	employee_detail（該当あり）／search_result（該当なし・0件）／search（社員番号が未入力のとき）
UC003	登録画面表示	GET	/input	showInputPage	input
UC003	登録画面へ戻る（修正）	POST	/input	backToInputPage	input
UC003	登録確認処理	POST	/inputConfirm	confirmRegistration	input_confirm（エラー時は input）
UC003	登録確定処理	POST	/saveEmployee	saveEmployee	input_complete

ユースケース	機能・アクション	HTTP	URLパス	コントローラーメソッド	遷移先ビュー名
UC004	変更画面表示	GET	`/changeInput/{empNo}`	`changeInput`	`change`
UC004	変更画面へ戻る（修正）	POST	`/changeInput/{empNo}`	`backToChangeInput`	`change`
UC004	変更確認処理	POST	`/changeConfirm`	`changeConfirm`	`change_confirm`（エラー時は `change`）
UC004	変更確定処理	POST	`/changeEmployee`	`changeEmployee`	`change_complete`
UC005	削除確認画面表示	GET	`/deleteConfirm/{empNo}`	`deleteConfirm`	`delete_confirm`
UC005	削除確定処理	POST	`/deleteEmployee`	`deleteEmployee`	`delete_complete`

> **HTTP メソッドの使い分け**：画面の初回表示・検索・削除確認画面の表示は **GET**、登録・変更の入力確認と確定、削除の確定、および確認画面からの「修正する」による入力画面への戻りは **POST** を用いる。特に `/input`・`/changeInput/{empNo}` は、**初回表示（GET）では DB の現在値（変更時）または空のフォームをセット**し、**確認画面からの「修正する」（POST）では利用者が入力した値を保持したまま再表示**する（DB の値で上書きしない）。ただし、登録・変更のパスワード欄は初回表示・戻り・入力エラー時とも値を表示せず、入力を求める。GET と POST はそれぞれ別のコントローラーメソッド（例：`changeInput` と `backToChangeInput`）で受ける。

各ユースケースの具体的なメソッド呼び出し、およびMVCモデル間（View, Controller, Service, Repository）のデータの流れを以下に定義する。

10.2 UC001: 社員一覧を表示する

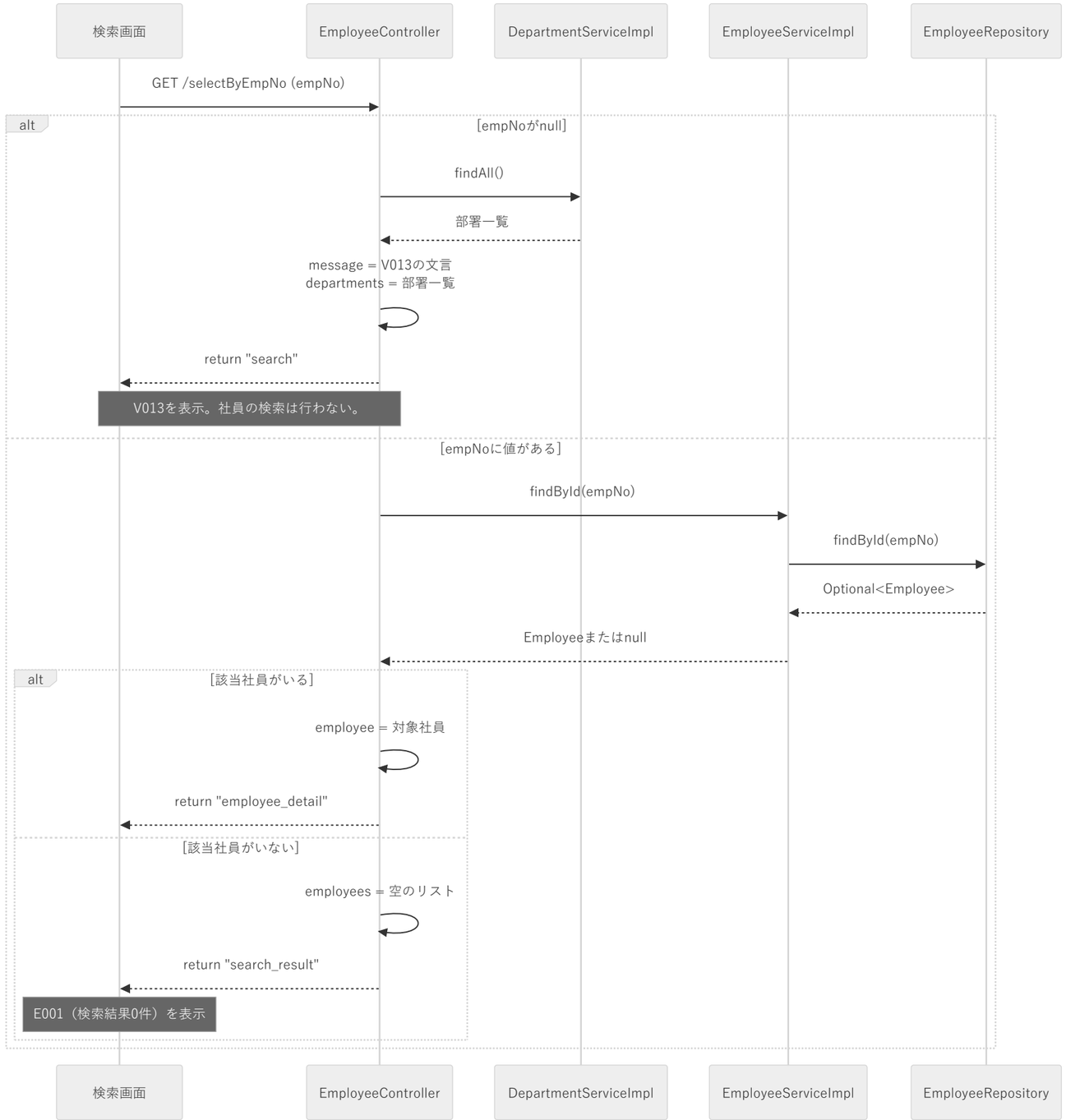


10.3 UC002: 社員情報を検索する

初回の `GET /search` は `showSearch(Model)` が受ける。`DepartmentServiceImpl.findAll()` から `DepartmentRepository.findAll()` を呼び、取得した部署一覧をModelの `departments` に設定して `search` を返す。

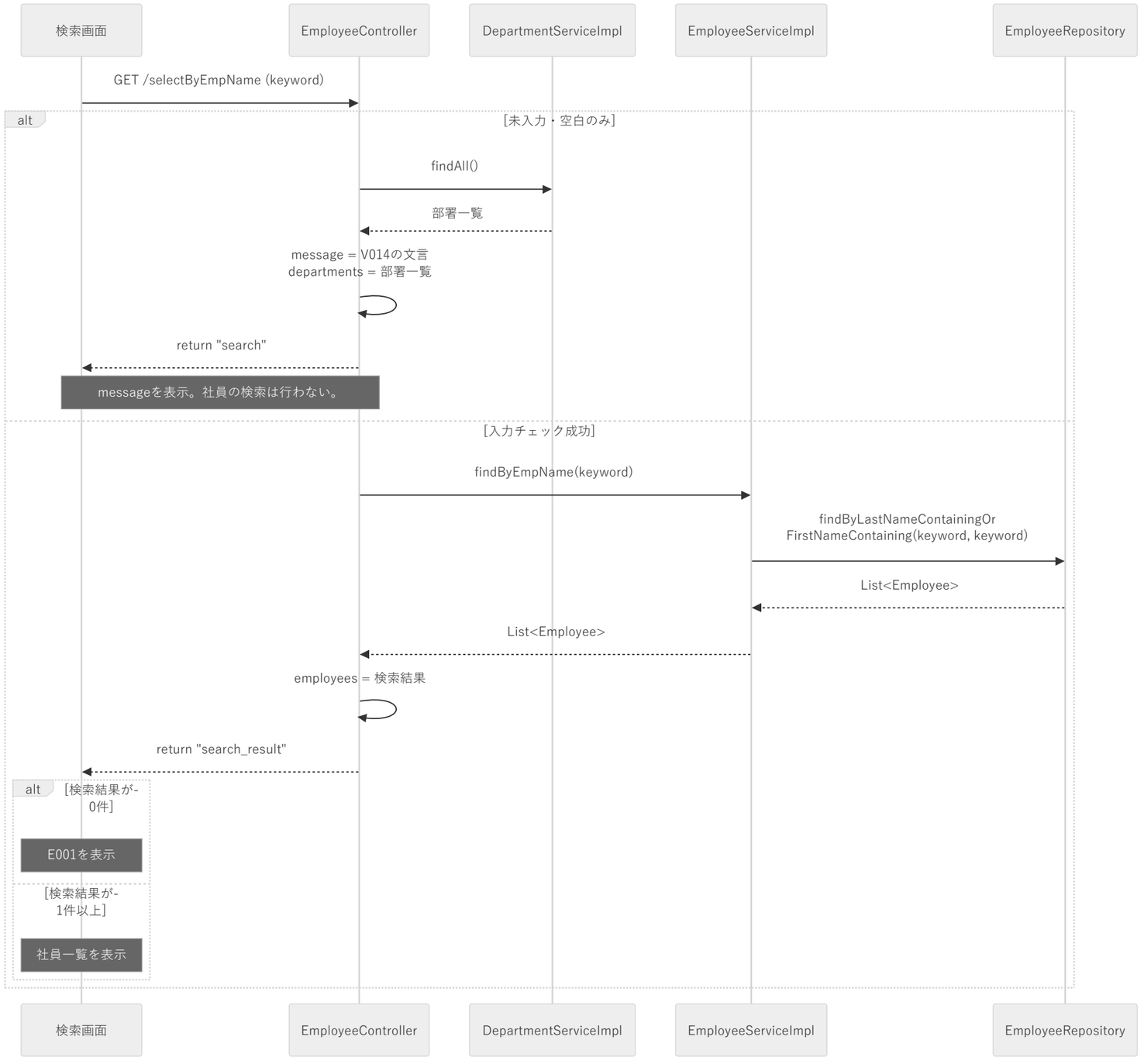
各検索URLは専用メソッドで受け、対応する条件だけを検証・検索する。以下の図中の `message`・`departments`・`employees`・`employee` は、`model.addAttribute(...)` で設定するModel属性である。入力エラー時の部署取得も、上記と同じService・Repositoryの経路を使う。メッセージ文言は12章を参照する。

社員番号検索：searchByEmpNo(Integer, Model)

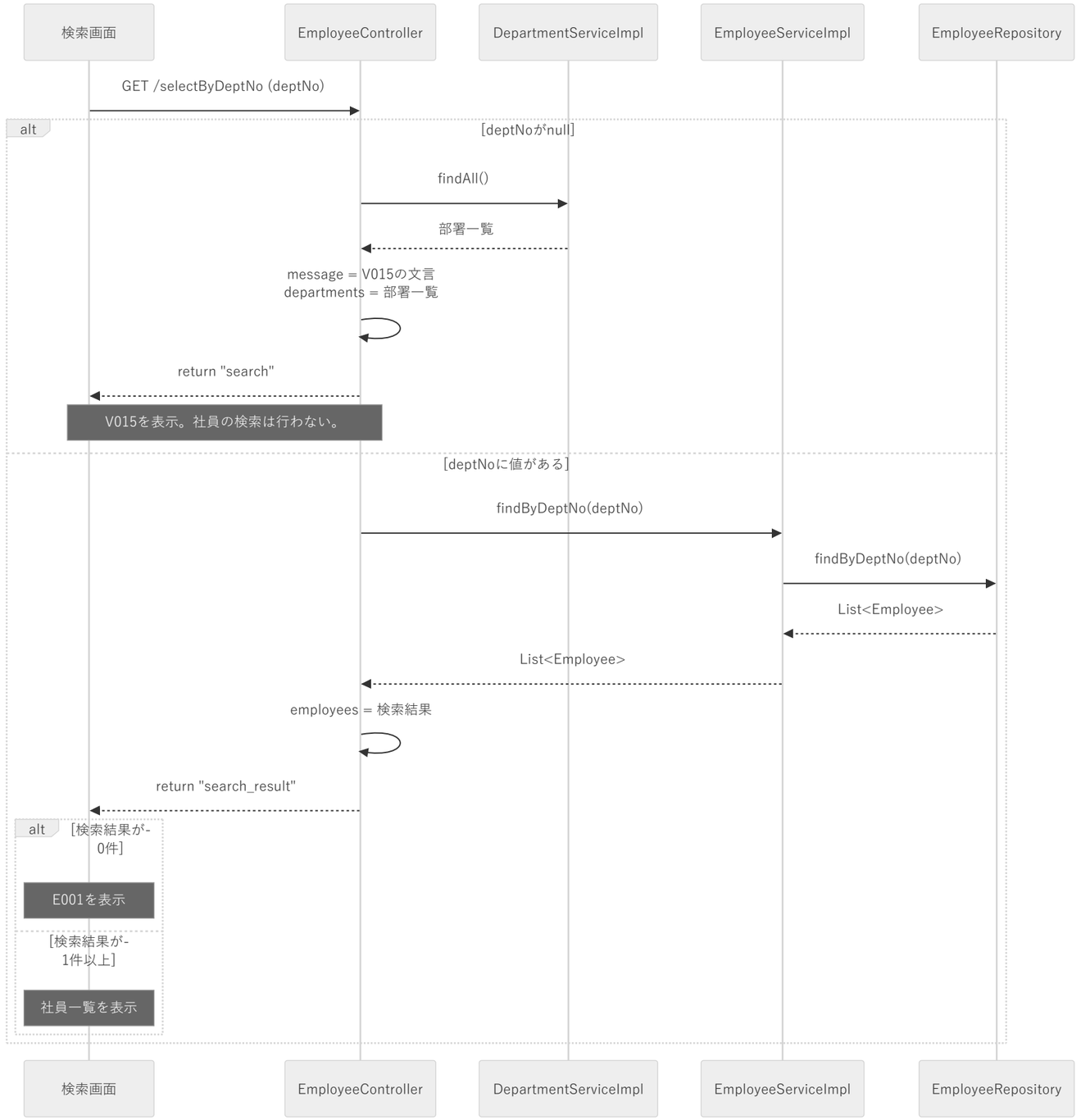


社員名検索：searchByEmpName(String, Model)

未入力（`null`・空文字・空白のみ）を判定する。検索キーワードの文字数上限は設けず、前後空白も除去しない。

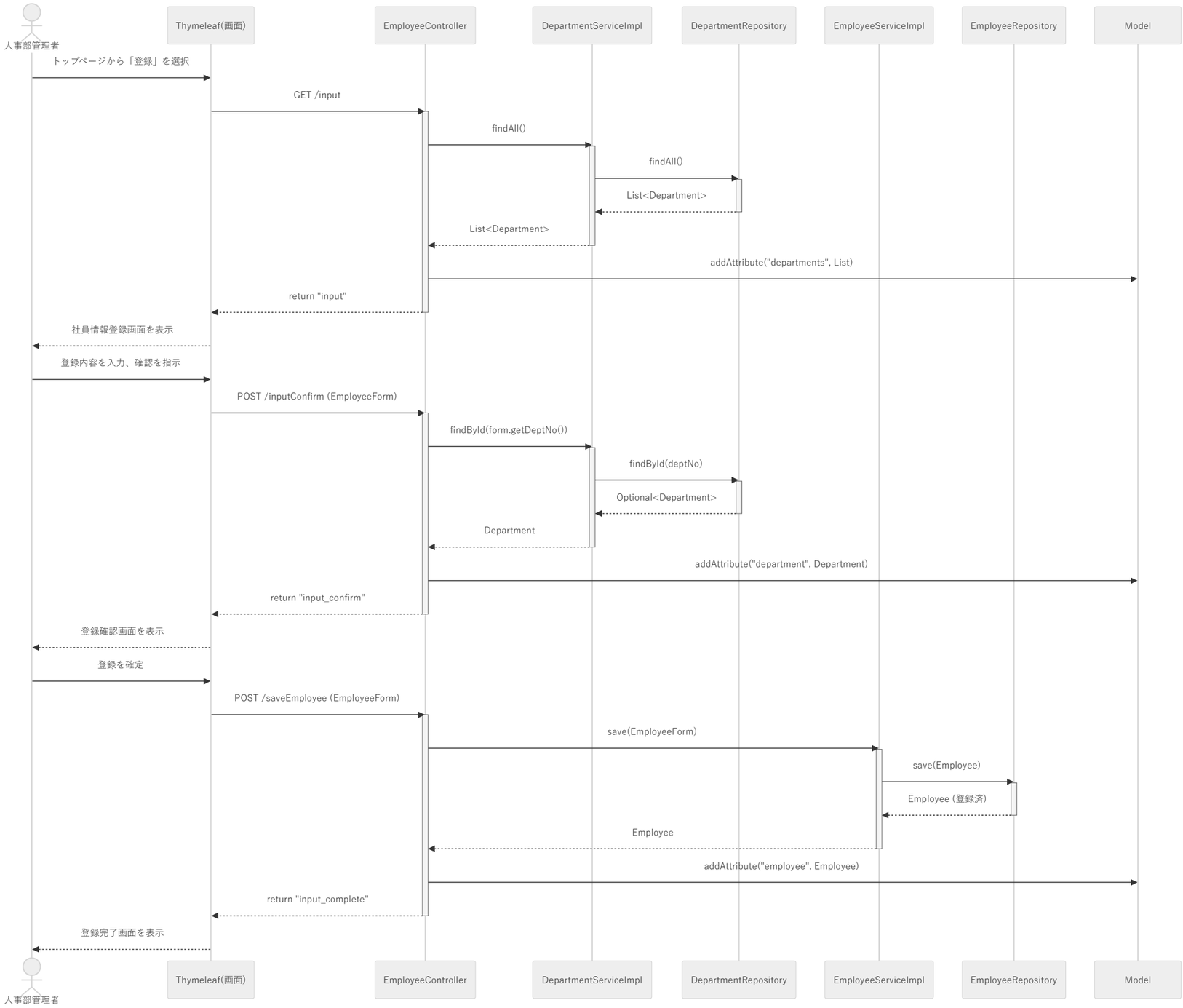


部署検索：searchByDeptNo(Integer, Model)

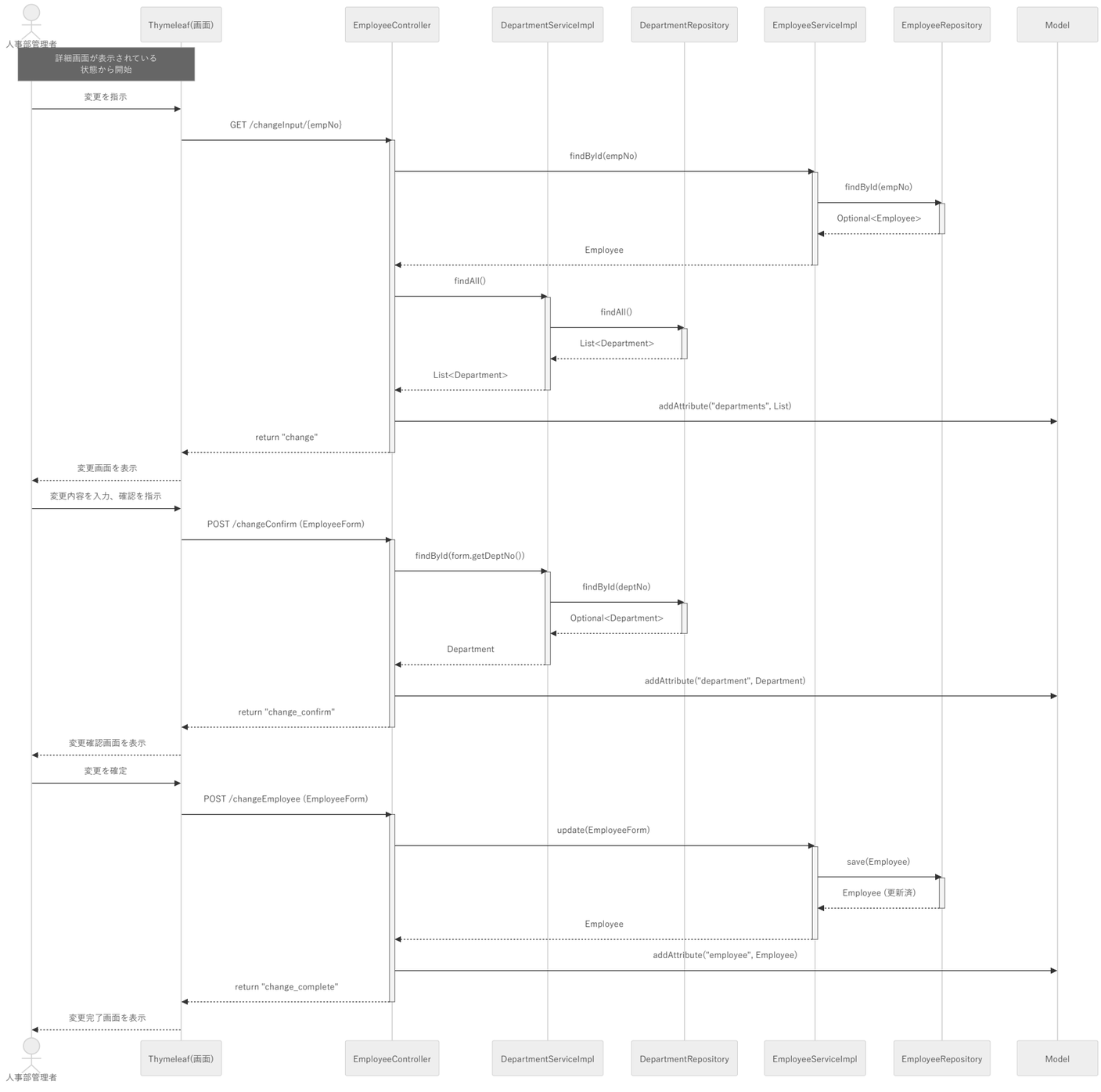


検索結果一覧の氏名リンクから `GET /detail/{empNo}` を送ると、`showDetail` が `EmployeeServiceImpl.findById(empNo)` を呼ぶ。ServiceはRepositoryから対象社員を取得し、ControllerはModelの `employee` に設定して `employee_detail` を返す。

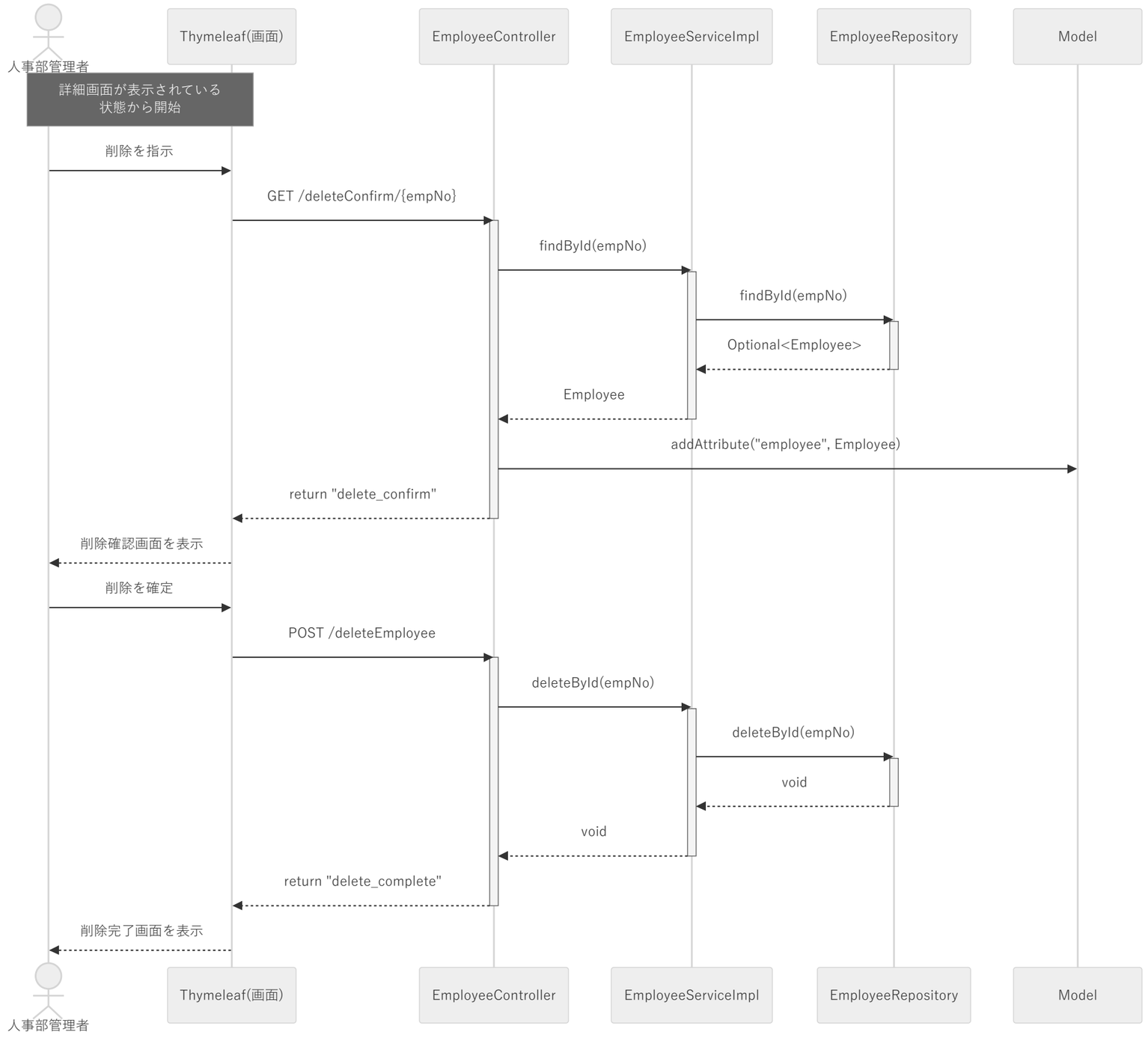
10.4 UC003: 社員情報を登録する



10.5 UC004: 社員情報を変更する



10.6 UC005: 社員情報を削除する



11. コンポーネントテスト仕様（任意）

本章では、余裕がある場合に実装するコンポーネントテスト（JUnit）の参考ケースを定義する。単体テストは必須課題ではないため、基本機能の実装とシステムテストを優先し、時間に余裕がある場合に取り組むこと。

テスト実行前の環境設定

コンポーネントテストを実行する前に、本番用データベース `eimsdb` とは別に、テスト用データベース `eimsdb_test` を構築する。`DBSET/EIMS_TEST_DBSET` にあるスクリプトを使用し、テスト用のユーザー、テーブルおよび初期データを準備すること。

テスト用の接続情報は `src/test/resources/application-test.properties` に定義する。この設定では接続先を `eimsdb_test` とし、`spring.jpa.hibernate.ddl-auto=validate` を使用する。テーブルはテスト実行時に自動作成されないため、`eimsdb_test` が未構築である場合や、テーブル定義とエンティティ定義が一致しない場合は、`mvn test` が失敗する。

テストクラスには `@ActiveProfiles("test")` を付与し、`application-test.properties` を有効にする。テストが起動しない場合は、次の順に確認すること。

1. `EIMS_TEST_DBSET` のスクリプトで `eimsdb_test` が構築済みであること
2. `application-test.properties` の接続先、ユーザー名、パスワードがテスト用DBの設定と一致していること
3. テストクラスに `@ActiveProfiles("test")` が付与されていること
4. `spring.jpa.hibernate.ddl-auto=validate` の検証エラーが発生していないこと

11.1 Repository テストケース（任意）（EmployeeRepository）

テストID	テスト対象メソッド	テスト内容	期待結果
CT-R-001	findById	存在する社員番号を指定して検索する。	指定した社員番号の社員情報が1件取得できること。
CT-R-002	findByLastNameContaining...	氏名の一部（漢字）を指定して検索する。	氏または名にキーワードを含むすべての社員が取得できること。
CT-R-003	findByDeptNo	存在する部署番号を指定して検索する。	その部署に所属するすべての社員が取得できること。

11.2 Controller テストケース（任意）（EmployeeController）

テストID	機能	テストケース	期待結果
CT-C-001	トップ	GET /index	index ビューが返されること。
CT-C-001-2	一覧	GET /employeeList	全社員リストが Model に設定され、employee_list ビューが返されること。

テストID	機能	テストケース	期待結果
CT-C-002	検索画面	GET /search	部署リストが Model に設定され、search ビューが返されること。
CT-C-003	氏名検索	GET /selectByEmpName (正常系)	検索キーワードに合致する社員リストが取得され、search_result ビューが返されること。
CT-C-004	氏名検索	GET /selectByEmpName (0件)	空のリストが返され、画面に「検索条件に一致する社員は見つかりませんでした。」と表示されること。
CT-C-005	番号検索	GET /selectByEmpNo (正常系)	指定した社員の詳細情報が Model に設定され、employee_detail ビューが返されること。
CT-C-005-2	詳細	GET /detail/{id}	指定した社員の詳細情報が Model に設定され、employee_detail ビューが返されること。
CT-C-006	番号検索	GET /selectByEmpNo (該当なし)	search_result ビューが返され、画面に「検索条件に一致する社員は見つかりませんでした。」と表示されること。
CT-C-006-2	番号検索	GET /selectByEmpNo (未送信・空文字・空白のみ（半角・全角))	社員を検索せず、部署リストとV013がModelに設定され、searchに「社員番号を入力してください。」と表示されること。
CT-C-007	登録画面	GET /input	部署リストが Model に設定され、input ビューが返されること。
CT-C-008	登録確認	POST /inputConfirm (正常)	入力内容が保持され、input_confirm ビューが返されること。
CT-C-009	登録確認	POST /inputConfirm (必須エラー)	必須項目未入力時に、input ビューに戻りエラーが表示されること。
CT-C-010	登録確認	POST /inputConfirm (サイズエラー)	文字数制限違反時に、input ビューに戻りエラーが表示されること。
CT-C-011	登録実行	POST /saveEmployee	社員情報が保存され、採番された社員番号とともに input_complete ビューが返されること。
CT-C-012	削除確認	GET /deleteConfirm/{id}	指定した社員情報が Model に設定され、delete_confirm ビューが返されること。
CT-C-012-2	削除確認	GET /deleteConfirm/{id} (該当なし)	search_result ビューが返され、削除できない旨のメッセージが表示されること。
CT-C-013	削除実行	POST /deleteEmployee	社員情報が削除され、delete_complete ビューが返されること。
CT-C-014	変更画面	GET /changeInput/{id}	既存の社員情報と部署リストが Model に設定され、change ビューが返されること。
CT-C-015	変更確認	POST /changeConfirm (正常)	入力内容が保持され、change_confirm ビューが返されること。
CT-C-016	変更確認	POST /changeConfirm (必須エラー)	必須項目未入力時に、change ビューに戻りエラーが表示されること。
CT-C-016-2	変更確認	POST /changeConfirm (サイズエラー)	文字数制限違反時に、change ビューに戻りエラーが表示されること。

テストID	機能	テストケース	期待結果
CT-C-017	変更実行	POST /changeEmployee	社員情報が変更され、change_complete ビューが返されること。
CT-C-018	部署検索	GET /selectByDeptNo（正常系）	指定した部署に所属する社員リストが取得され、search_result ビューが返されること。
CT-C-019	氏名検索	GET /selectByEmpName（未送信・空文字・空白のみ）	社員を検索せず、部署リストとV014がModelに設定され、searchに「社員名を入力してください。」と表示されること。
CT-C-020	部署検索	GET /selectByDeptNo（未送信・空文字・空白のみ）	社員を検索せず、部署リストとV015がModelに設定され、searchに「部署を選択してください。」と表示されること。
CT-C-021	氏名検索	GET /selectByEmpName（11文字・256文字）	長いキーワードも拒否・切り詰めをせず、そのまま検索へ渡してsearch_resultを表示すること。該当社員がなければ0件を表示すること。
CT-C-022	検索分離	各検索URLへ他の検索条件も送信する	URLに対応する条件だけを使い、他の検索へ切り替わらないこと。対象項目が未入力なら対応するメッセージを表示すること。
CT-C-023	氏名検索	前後に空白を含む条件を送信する	空白を除去せず、入力値をそのまま検索に渡すこと。

> ※ 未入力の判定は3つの検索メソッドがそれぞれ行うため、社員番号のケースだけで代表せず、全検索のメッセージ表示と「社員の検索を呼ばないこと」を個別に確認する。コンポーネントテストの実装は任意だが、検索の入力チェック自体は標準・簡略実装とも必須である。

12. 共通メッセージ一覧

システム内で使用する定型メッセージを以下に定義する。

メッセージID	内容	用途
M001	登録が完了しました	登録完了画面
M002	新しい社員情報が正常にデータベースに保存されました。	登録完了画面
M003	変更が完了しました	変更完了画面
M004	社員情報が正常に変更されました。	変更完了画面
M005	削除が完了しました	削除完了画面
M006	社員情報がシステムから正常に削除されました。	削除完了画面
C001	以下の内容で登録します。よろしいですか？	登録確認画面
C002	以下の内容に変更します。よろしいですか？	変更確認画面
C003	この社員情報を削除します。この操作は取り消せません。よろしいですか？	削除確認画面
E001	検索条件に一致する社員は見つかりませんでした。	検索結果（0件時）
E002	指定された社員情報は存在しないため、削除できません。	削除確認・削除実行時の対象なし
V001	氏は必須項目です	氏の未入力
V002	氏は10文字以内で入力してください	氏の文字数超過
V003	名は必須項目です	名の未入力
V004	名は10文字以内で入力してください	名の文字数超過
V005	氏（カナ）は必須項目です	氏（カナ）の未入力
V006	氏（カナ）は20文字以内で入力してください	氏（カナ）の文字数超過
V007	名（カナ）は必須項目です	名（カナ）の未入力
V008	名（カナ）は20文字以内で入力してください	名（カナ）の文字数超過
V009	パスワードは必須項目です	パスワードの未入力
V010	パスワードは4文字以上16文字以内で入力してください	パスワードの文字数不足・超過
V011	性別は必須項目です	性別の未選択
V012	部署は必須項目です	部署の未選択
V013	社員番号を入力してください。	社員番号検索の未入力
V014	社員名を入力してください。	社員名検索の未入力
V015	部署を選択してください。	部署検索の未選択

13. 発展課題（任意）

本章の課題は必須ではない。基本機能（登録・変更・削除・検索）の実装とシステムテストを優先し、**時間に余裕がある場合**に取り組むこと。

13.1 二重ポスト（ダブルサブミット）対策

何が問題か

登録・変更・削除の **完了画面が表示された状態で、ブラウザの「更新（リロード）」ボタンを押したり、「戻る」→「進む」を行うと、直前の送信（POST）がもう一度実行されてしまう** ことがある。たとえば登録完了画面で更新を押すと、**同じ社員がもう 1 件登録される** といった事象が起こり得る。これを **二重ポスト（ダブルサブミット）** と呼ぶ。

本演習の基本機能では、この対策は **必須ではない**（確認画面を挟むことで誤操作自体は減らしている）。以下は、仕組みを理解したうえで対策を実装してみたい場合の発展課題である。

対策（PRG パターン）

完了画面を「POST の処理結果としてそのまま表示する」のではなく、**いったん完了画面表示用の GET の URL へリダイレクト（`redirect:`）してから表示する** ように作り替える。この「Post → Redirect → Get」の流れを **PRG パターン** と呼ぶ。

- `redirect:` の書き方は、既習の「Spring Boot 入門」研修で学習済みである。
- リダイレクト先の GET で完了画面に表示したい情報（登録した社員の情報など）は、`RedirectAttributes`（フラッシュスコープ）などを使って引き継ぐ。

なぜ二重ポストを防げるのか：リダイレクトを行うと、ブラウザに表示される URL が「完了画面を表示する GET の URL」に変わる。そのため、利用者が更新ボタンを押しても **再実行されるのは GET（完了画面の再表示）だけ** となり、登録・変更・削除を行う POST は再送されない。

13.2 社員一覧の並び替え（ソート）

本演習の基本仕様では、社員一覧の**表示順は保証していない**（データベースが返した順で表示する）。そのため、登録や削除を行うと一覧の並びが直感的でなくなることがある。

発展として、一覧を**一定の順序（例：社員番号の昇順）で安定して表示**するよう改良する。

- リポジトリのメソッド名クエリで並び順を指定できる（例：`findAllByOrderByEmpNoAsc()`）。あるいは `findAll(Sort.by("empNo"))` のように `Sort` を渡す方法もある。
- メソッド名クエリ（`findBy…` / `…OrderBy…`）の書き方は、既習の「Spring Boot 入門」研修で学習済みである。